

PES UNIVERSITY, BANGALORE

Department of Computer Science and Engineering

Software Requirements Specification for BloodConnect

Project ID	BBMS-2026-T5
Group	Team 5
Customer / Stakeholder	Blood Bank Administrator / Partner Hospital Coordinator
Course	UE24CS341A - Software Engineering, V Semester
Development Methodology	Agile (6 sprints)
Prepared by	Ankitha Lakshmmi Mohan (PES1UG24CS067), Anumula Arya (PES1UG24CS072), Arnav Chaudhary (PES1UG24CS081), Ch Yashwitha (PES1UG24CS121)

Table of Contents

1. Introduction.....	3
1.1 Purpose.....	3
1.2 Intended Audience and Reading Suggestions	3
1.3 Product Scope.....	3
1.4 References	4
2. Overall Description.....	4
2.1 Product Perspective.....	4
2.2 Product Functions.....	4
2.3 Team Profile and Ownership	5
2.4 User Classes and Characteristics	5
2.5 Operating Environment	5
2.6 Design and Implementation Constraints.....	6
2.7 Assumptions and Dependencies	6
2.8 Plan of Work	6
3. External Interface Requirements	7
4. Analysis Models.....	7
4.1 Use Case Diagram	7
4.2 Domain / Entity-Relationship Model	8
5. System Features	9
6. Other Nonfunctional Requirements.....	11
7. Other Requirements.....	12
Appendix A: Glossary	12
Appendix B: Field Layouts	13
Appendix C: Requirement Traceability Matrix.....	13

1. Introduction

1.1 Purpose

This document is the Software Requirements Specification (SRS) for BloodConnect, a web-based system that Team 5 is building to manage blood donors, component-wise inventory and blood requests for a blood bank and its partner hospitals. It is Version 1.1, and it revises the earlier draft so that the scope, team ownership and plan of work match the approved project synopsis exactly. It sets out what the system does, who uses it, and the functional and non-functional requirements that the design and test phases of the Level 3 Software Engineering mini-project (UE24CS341A) will be measured against.

1.2 Intended Audience and Reading Suggestions

The primary readers of this document are the four members of Team 5, the course faculty who evaluate the project, and the blood bank and hospital stakeholders the system is being built for. A reader who only wants the big picture can stop after Sections 1 and 2. Section 3 covers how the system talks to its users and to other software, Section 4 gives the use-case and domain diagrams that Section 5's requirements are drawn from, and Section 6 lists the non-functional, safety and security requirements. Section 7 and the appendices are reference material - the glossary, the database field layouts and the requirement traceability matrix - and are meant to be consulted rather than read straight through.

1.3 Product Scope

Blood banks and partner hospitals today rely on manual registers, spreadsheets or disconnected local tools to track donors, stock and requests. That makes it hard to find compatible blood quickly during an emergency, gives no real-time visibility into stock across components such as plasma and platelets, lets donor eligibility checks and unit expiries slip through unnoticed, and leaves hospitals without a central place to raise and follow up urgent requests. BloodConnect addresses this by digitising donor registration and eligibility screening, component-wise inventory tracking with expiry alerts, the blood request/issue workflow, appointment and camp scheduling, and administrator reporting, with the aim of improving blood availability, cutting wastage from expiry, and shortening request turnaround time.

For this phase, the project is limited to a single blood bank branch. Three items were explicitly agreed to be out of scope: payment or billing integration, integration with any external national blood-registry API, and native mobile applications - the product is web-responsive only. These exclusions keep the six-sprint schedule realistic and are recorded here so that they are not accidentally treated as missing requirements during evaluation.

1.4 References

- BloodConnect - Project Synopsis, Team 5, submitted as Deliverable 1.
- PES University, Department of CSE - Software Engineering (UE24CS341A) Level 3 Mini-Project guidelines.

2. Overall Description

2.1 Product Perspective

BloodConnect is a new, self-contained product rather than an add-on to an existing system, and it is meant to replace the manual, spreadsheet-driven process the target blood bank and its partner hospitals use

today. It is built around a REST API that is consumed by four role-based web views - Donor, Blood Bank Staff, Hospital/Requester and Administrator - and it depends on an email/SMS channel, mocked during development, to deliver reminders and alerts. Section 4.1 shows the use-case diagram for these four actors, and Section 4.2 shows the domain model underneath.

2.2 Product Functions

At a high level, BloodConnect provides:

- Donor registration, login and eligibility screening (age, weight, minimum interval since the last donation).
- Component-wise blood inventory management - whole blood, plasma, platelets and RBC - with automatic expiry tracking.
- Automated low-stock and near-expiry alerts so staff can act before a shortage or a wasted unit.
- Blood request submission by hospitals or patients with an urgency level, plus a staff approval/issue workflow.
- Donation camp and individual appointment scheduling.
- Notifications to donors, staff and requesters for reminders, alerts and status changes.
- An administrator dashboard and basic reporting on donations, stock and requests.

2.3 Team Profile and Ownership

Each member of Team 5 owns one complete, end-to-end functional area of BloodConnect, as agreed in the synopsis. UI design, database work and testing are listed as secondary responsibilities because they support the owned feature - they are not a substitute for owning one.

Member	USN / SRN	Owned Functionality	Secondary Responsibility
Ankitha Lakshmmi Mohan	PES1UG24CS067 / PES1202402195	Donor registration, eligibility screening & donor search	API review
Anumula Arya	PES1UG24CS072 / PES1202400180	Appointment scheduling, notifications & admin/reporting	Testing/CI
Arnav Chaudhary	PES1UG24CS081 / PES1202400202	Blood request & issue management workflow	Database
Ch Yashwitha	PES1UG24CS121 / PES1202403018	Blood inventory & stock management (component-wise, expiry tracking)	UI/UX

2.4 User Classes and Characteristics

User Class	Privileges	Characteristics
Donor	Register, log in, complete eligibility screening, view donation history, book appointments, receive notifications	Non-technical, occasional use, mostly on a phone or browser
Blood Bank Staff	Search donors, manage inventory, review/approve/reject requests, issue blood, manage camps	Trained operational users, on the system daily
Hospital / Requester	Submit blood requests with an urgency level, track request status	Semi-technical, often working under time pressure during an emergency

User Class	Privileges	Characteristics
Administrator	Manage users and roles, configure thresholds, view dashboards and reports, manage camps	Higher privilege, uses the system less often but for critical configuration

2.5 Operating Environment

BloodConnect is a responsive web application meant to run in modern desktop and mobile browsers such as Chrome, Firefox and Edge. The backend is built on Python with Django and Django REST Framework, backed by MySQL or PostgreSQL, and a Node.js service handles real-time notifications and alerts alongside it. The whole stack is containerised with Docker so it behaves the same way in development, CI and staging.

2.6 Design and Implementation Constraints

- The project follows Agile across six sprints, tracked with GitHub/Jira (or an equivalent), as committed in the synopsis.
- A CI/CD pipeline, code review and basic security checks are expected to run before each milestone is submitted.
- The technology stack is fixed: Django/DRF for the core backend, Node.js for notifications, MySQL/PostgreSQL for storage and Docker for containerisation.
- Donor health and eligibility data is sensitive and must stay behind role-based access control.
- Each member owns one complete feature end-to-end; UI, database and testing work are supporting activities and cannot be substituted as a member's primary ownership.

2.7 Assumptions and Dependencies

- Reliable internet connectivity is assumed at the blood bank and at partner hospital sites.
- Donors are assumed to self-report health information honestly at registration, subject to a final check by staff at the time of donation.
- The system depends on a third-party or mocked SMS/email gateway for notifications during development and testing.
- This phase assumes a single blood bank branch; supporting multiple branches is out of scope for now.
- Shelf-life rules per blood component are assumed to be configurable rather than hard-coded into the application.

2.8 Plan of Work

The synopsis lays out a six-sprint schedule, each ending in a working, demo-able increment. That plan is reproduced here so the requirements in Section 5 can be traced back to the sprint each is expected to land in.

Sprint	Planned Output	Acceptance Evidence
1	Project setup, requirements baseline, donor login	User stories, Git repo, working login
2	Donor registration, eligibility screening, donor search	Demo + unit tests
3	Blood inventory intake, stock levels, expiry tracking	Integration tests + PRs
4	Blood request submission, approval and issue workflow	System demo + test evidence

Sprint	Planned Output	Acceptance Evidence
5	Appointment scheduling, notifications, admin/reporting	Validation + coverage report
6	System validation and release	Final report + deployment/demo

Agile suits this project because requirements from blood bank staff and hospital users are expected to be refined as the team learns more, each sprint's module lines up with one member's end-to-end ownership, and short feedback loops reduce the risk of integrating the wrong workflow late in the project.

3. External Interface Requirements

3.1 User Interfaces

BloodConnect provides three role-based web interfaces: a Donor portal (registration, eligibility status, appointment booking, notifications), a Staff/Admin dashboard (inventory, requests, camps, reports), and a Hospital/Requester portal (request submission and tracking). All three share a consistent layout, standard navigation and inline form validation, and use toast/alert components for low-stock, near-expiry and request-status notifications. A calendar-style widget is used wherever a donor or a camp organiser needs to pick a date or a slot.

3.2 Software Interfaces

The backend exposes a JSON REST API built with Django REST Framework, which the web front end consumes. A separate Node.js service talks to that backend to dispatch notifications. Data is persisted in MySQL or PostgreSQL through the Django ORM, and Postman is used for API testing during development and in CI.

3.3 Communications Interfaces

All client-server traffic goes over HTTPS. Notifications go out over SMTP for email and through an SMS gateway (or a mocked equivalent while the gateway isn't wired up yet). JSON is used as the data-interchange format for every API request and response.

4. Analysis Models

4.1 Use Case Diagram

The diagram below shows the four actors from the synopsis - Donor, Blood Bank Staff, Hospital/Requester and Administrator - and how each one interacts with BloodConnect. The <<include>> relationships capture that submitting a request always leads into its later approval or rejection, and that an approval always leads into the issue step that follows it.

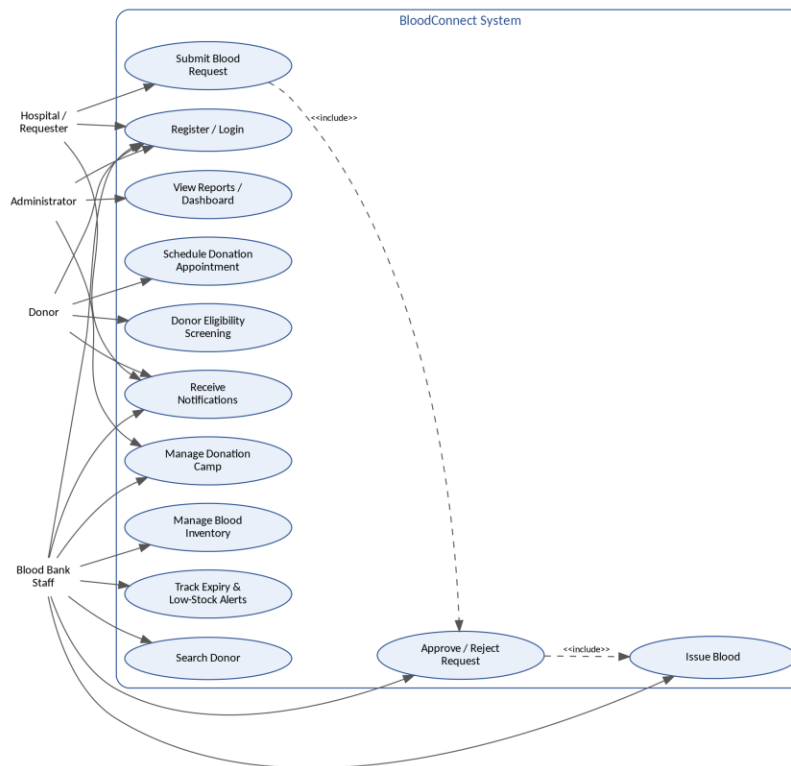


Figure 4.1 - BloodConnect Use Case Diagram

UC ID	Use Case	Primary Actor	Outcome
UC-01	Register / Login	Donor / Staff / Hospital / Admin	Authenticated session created
UC-02	Donor Eligibility Screening	Donor	Eligibility status recorded
UC-03	Search Donor	Staff	Matching donor list displayed
UC-04	Schedule Donation Appointment	Donor	Appointment created
UC-05	Manage Blood Inventory	Staff	Inventory record added/updated
UC-06	Track Expiry & Low-Stock Alerts	Staff	Alert generated
UC-07	Submit Blood Request	Hospital / Requester	Request created and routed to staff
UC-08	Approve / Reject Request	Staff	Request status updated
UC-09	Issue Blood	Staff	Issue record created; stock decremented
UC-10	Manage Donation Camp	Staff / Admin	Camp published or updated
UC-11	Receive Notifications	Donor / Staff / Hospital	Notification delivered
UC-12	View Reports / Dashboard	Administrator	Report/dashboard displayed

4.2 Domain / Entity-Relationship Model

The diagram below shows BloodConnect's main entities and how they relate to each other. It underpins the field layouts in Appendix B and will be carried into the physical database design in the next phase.

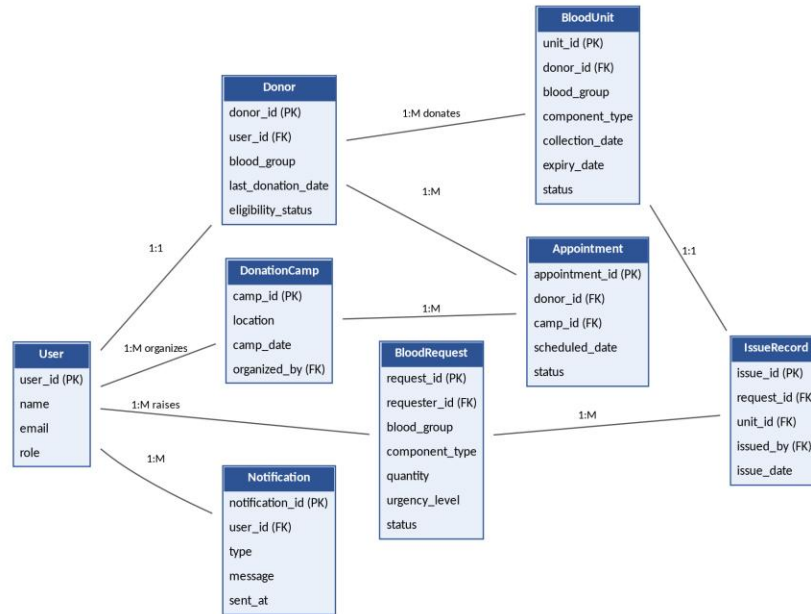


Figure 4.2 - BloodConnect Domain / Entity-Relationship Diagram

5. System Features

5.1 Donor Registration & Eligibility Management

Owned by: Ankitha Lakshmmi Mohan.

5.1.1 Description and Priority

High priority. This feature lets donors register and log in themselves, then screens their eligibility against configurable rules - age, weight and the minimum gap since their last donation - before they can book a donation appointment. It also gives staff a way to search the registered donor base.

5.1.2 Stimulus/Response Sequences

- Donor fills in the registration form; the system validates the input, creates the account and sends a confirmation notification.
- Donor or staff enters login credentials; the system authenticates them and opens a role-based session.
- Donor starts eligibility screening; the system checks the configured rules and returns an eligible/not-eligible result with a reason.
- Staff searches by blood group, location or eligibility status; the system returns the matching donor list.

5.1.3 Functional Requirements

ID	Requirement	Priority	Verification
FR-001	The system shall allow a new donor to register using a unique email address and mobile number.	High	Functional test
FR-002	The system shall authenticate a donor, staff, hospital/requester or administrator using email and password before granting access.	High	Functional + security test

ID	Requirement	Priority	Verification
FR-003	The system shall screen donor eligibility against configurable rules (e.g. age 18-65, weight $\geq$ 50 kg, minimum 90-day gap since last donation).	High	Boundary + functional test
FR-004	The system shall display the eligibility result to the donor, including the reason when the donor is found ineligible.	High	Functional test
FR-005	The system shall allow staff to search registered donors by blood group, location and eligibility status.	Medium	Functional test

5.2 Blood Inventory & Expiry Management

Owned by: Ch Yashwitha.

5.2.1 Description and Priority

High priority. This feature keeps a component-wise blood inventory - whole blood, plasma, platelets and RBC - works out expiry dates automatically, and proactively warns staff about units nearing expiry or stock running low, so wastage stays down and shortages get caught early.

5.2.2 Stimulus/Response Sequences

- Staff records a newly donated unit; the system creates a BloodUnit record with a computed expiry date and updates the stock count.
- A scheduled check runs against current inventory; the system flags units nearing expiry and notifies staff.
- Available stock for a blood group/component drops below its configured threshold; the system raises a low-stock alert.

5.2.3 Functional Requirements

ID	Requirement	Priority	Verification
FR-006	The system shall allow staff to record a new blood unit with blood group, component type, collection date and quantity.	High	Functional + integration test
FR-007	The system shall automatically calculate the expiry date of a blood unit based on component-specific shelf-life rules.	High	Unit + integration test
FR-008	The system shall maintain a real-time, component-wise stock count for each blood group.	High	Integration + system test
FR-009	The system shall flag blood units nearing expiry (within a configurable threshold, e.g. 3 days) and notify staff.	High	Integration test
FR-010	The system shall generate a low-stock alert when available quantity of a blood group/component falls below the configured minimum threshold.	High	Integration test
FR-011	The system shall prevent an expired or discarded blood unit from being selected for issue.	High	Negative integration test

5.3 Blood Request & Issue Management

Owned by: Arnav Chaudhary.

5.3.1 Description and Priority

High priority. This feature lets hospitals or patients submit blood requests with an urgency level, and lets staff review, approve or reject those requests and then issue matching, non-expired blood units against the ones that are approved.

5.3.2 Stimulus/Response Sequences

- Hospital/requester submits a request; the system validates it, records it and notifies staff.
- Staff reviews a pending request and approves or rejects it, giving a reason if it's rejected.
- Staff issues blood against an approved request; the system deducts inventory, creates an issue record and updates the request status.

5.3.3 Functional Requirements

ID	Requirement	Priority	Verification
FR-012	The system shall allow a hospital/requester to submit a blood request specifying blood group, component type, quantity and urgency level.	High	Functional test
FR-013	The system shall route newly submitted requests to staff for review and display pending requests ordered by urgency level.	High	System test
FR-014	The system shall allow staff to approve or reject a request, requiring a reason when the request is rejected.	High	Functional test
FR-015	The system shall create an issue record only against an approved request and a matching, non-expired blood unit.	High	Integration test
FR-016	The system shall automatically decrement the corresponding inventory stock when a blood unit is issued.	High	Integration test
FR-017	The system shall track and display request status as SUBMITTED, APPROVED, REJECTED, ISSUED or CLOSED.	High	System test

5.4 Appointment Scheduling, Notifications & Admin Reporting

Owned by: Anumula Arya.

5.4.1 Description and Priority

Medium-to-high priority. This feature lets donors book a donation appointment or a camp slot, sends out automated notifications - reminders, low-stock and near-expiry alerts - and gives administrators a way to manage camps and see operational dashboards and reports.

5.4.2 Stimulus/Response Sequences

- Donor picks an available camp/slot; the system checks capacity and creates the appointment.
- An appointment date approaches; the system sends the donor a reminder notification.
- Admin/staff schedules a donation camp; the system publishes it so donors can book it.
- Admin asks for a report; the system pulls together donation, stock and request data and displays it.

5.4.3 Functional Requirements

ID	Requirement	Priority	Verification
FR-018	The system shall allow a donor to book an available appointment slot for a donation camp or walk-in donation.	High	Functional test

ID	Requirement	Priority	Verification
FR-019	The system shall prevent a slot from being booked beyond the configured capacity of a camp.	Medium	Boundary + concurrency test
FR-020	The system shall send an automated reminder notification to a donor at least 24 hours before a scheduled appointment.	Medium	Notification integration test
FR-021	The system shall allow staff/administrators to create, update and cancel a donation camp with date, location and capacity.	Medium	Functional test
FR-022	The system shall provide an administrator dashboard summarising current stock levels, pending requests and upcoming camps.	Medium	System test
FR-023	The system shall allow an administrator to generate a report of donations, issues and wastage over a selected date range.	Low	Functional + report test

6. Other Nonfunctional Requirements

6.1 Non-Functional Requirements

ID	Category	Measurable Requirement	Verification
NFR-001	Performance	95% of inventory/search API requests shall receive a response within 2 seconds under the defined test load.	Performance test
NFR-002	Availability	The application shall be available at least 99% during the defined operating window, excluding planned maintenance.	Operational availability review
NFR-003	Usability	A new donor shall be able to complete registration in no more than 5 primary screens.	Usability walkthrough
NFR-004	Reliability	A blood unit shall never be issued against two different approved requests simultaneously.	Concurrency + integration test

6.2 Safety Requirements

- Expired or quarantined blood units must never be selectable for issue; this is enforced at both the UI and API layer.
- A donor flagged ineligible must be blocked from completing a donation-linked appointment until re-screened.
- Critical stock and expiry alerts must go out over more than one channel (in-app plus email/SMS) so an alert is less likely to be missed.

6.3 Security Requirements

ID	Security Requirement
SEC-001	Passwords shall be stored using a salted hash, never in plaintext.
SEC-002	Role-based access control shall prevent donors and hospital users from accessing staff/admin functions.
SEC-003	The system shall validate and sanitize all user-supplied fields before processing.
SEC-004	Authenticated APIs shall reject expired or invalid session tokens.
SEC-005	Donor medical/health information shall be accessible only to authorized staff and the donor themselves.
SEC-006	Security-relevant events such as failed login and unauthorized access attempts shall be logged.

6.4 Software Quality Attributes

Maintainability and reliability matter more here than ease-of-learning, since bad inventory or request-status data has real consequences outside the software. The system should stay adaptable - eligibility rules and stock thresholds are configuration, not hard-coded values - and portable, since Docker is what lets it run the same way across development, CI and staging.

6.5 Business Rules

- Only a screened, eligible donor's unit may be added to inventory.
- A blood unit past its expiry date cannot be issued and must be marked EXPIRED/DISCARDED.

- Only staff can move a request from SUBMITTED to APPROVED or REJECTED, and only an approved request may be issued.
- Only administrators can create or cancel donation camps and manage user roles.
- An appointment cannot be booked beyond a camp's configured capacity.

7. Other Requirements

- Database: see Appendix B for the field layouts behind the schema; referential integrity between Donor, BloodUnit, Request and IssueRecord is enforced at the database level.
- Data retention: donor health/eligibility records are kept in line with applicable data-protection practice; the exact deletion/anonymisation policy is left to be finalised during Design.
- Localisation: English only for this phase, but the schema is not built in a way that rules out adding another language later.
- Reuse: the authentication and role-based-access-control scaffolding that Django ships with may be reused instead of built from scratch.

Appendix A: Glossary

- Donor - a registered individual who donates blood and is subject to eligibility screening.
- Blood Unit - a single collected and tracked quantity of a blood component from one donation.
- Component - a specific blood product derived from whole blood (e.g. Plasma, Platelets, RBC).
- Eligibility Screening - the rule-based check performed before a donor is allowed to donate.
- Cross-match - compatibility verification between a blood unit and a recipient/request before issue.
- RBC - Red Blood Cells, a blood component.
- Camp - a scheduled blood donation drive/event at which donors can book appointments.
- Issue Record - the record created when a blood unit is dispatched against an approved request.
- RBAC - Role-Based Access Control.
- FR / NFR - Functional Requirement / Non-Functional Requirement.

Appendix B: Field Layouts

A sample field layout for the core entities from Section 4.2. This will be extended into a full schema during the Design phase.

Field	Data Type	Description	Mandatory
donor_id	UUID/String	Unique donor identifier	Y
email	String(100)	Unique login email	Y
blood_group	Enum	A+, A-, B+, B-, O+, O-, AB+, AB-	Y
unit_id	UUID/String	Unique blood unit identifier	Y
component_type	Enum	Whole Blood / Plasma / Platelets / RBC	Y
expiry_date	Date	Computed expiry date of the unit	Y
request_id	UUID/String	Unique blood request identifier	Y

Field	Data Type	Description	Mandatory
urgency_level	Enum	Routine / Urgent / Emergency	Y
status	Enum/String	Current state of the entity (request/unit/appointment)	Y

Appendix C: Requirement Traceability Matrix

A representative subset of requirements is shown below. Design and test references are marked TBD at this stage and will be filled in progressively during Design and during Implementation & Testing.

Req ID	Brief Description	Architecture Ref	Design Ref	Code/File Ref	Test Case ID	System Test ID
FR-001	Donor registration	TBD - Design Phase	TBD - Design Phase	TBD - Implementation Phase	TBD - Test Phase	TBD - System Validation
FR-006	Record new blood unit	TBD - Design Phase	TBD - Design Phase	TBD - Implementation Phase	TBD - Test Phase	TBD - System Validation
FR-012	Submit blood request	TBD - Design Phase	TBD - Design Phase	TBD - Implementation Phase	TBD - Test Phase	TBD - System Validation
FR-015	Issue against approved request	TBD - Design Phase	TBD - Design Phase	TBD - Implementation Phase	TBD - Test Phase	TBD - System Validation
FR-018	Book donation appointment	TBD - Design Phase	TBD - Design Phase	TBD - Implementation Phase	TBD - Test Phase	TBD - System Validation
SEC-002	Role-based access control	TBD - Design Phase	TBD - Design Phase	TBD - Implementation Phase	TBD - Test Phase	TBD - System Validation
NFR-001	API response time	TBD - Design Phase	TBD - Design Phase	TBD - Implementation Phase	TBD - Test Phase	TBD - System Validation

Software Architecture and Design Specification

Project: BloodConnect - Blood Bank Donor, Inventory & Request Management System

Project ID: BBMS-2026-T5 | Group: Team 5

Course: UE24CS341A - Software Engineering | V Semester

Version: 1.1 | Date: 02-10-2026 | Status: Draft for team review

Development Methodology: Agile (6 sprints)

Authors: Ankitha Lakshmmi Mohan (PES1UG24CS067); Anumula Arya (PES1UG24CS072); Arnav Chaudhary (PES1UG24CS081); Ch Yashwitha (PES1UG24CS121)

Revision History

Version	Date	Author	Change Summary
1.0	01-10-2026	Team 5	Initial working draft
1.1	02-10-2026	Team 5	Template-aligned revision with clearer wording, UML diagrams and open design issues.

Approvals

Role	Name	Signature/Date
Team review	All Team 5 members	
Course faculty	To be filled	
Stakeholder review	Blood Bank Administrator / Partner Hospital Coordinator	

1. Introduction

1.1 Purpose

This document defines BloodConnect’s planned components, data model, interfaces, workflows, and requirement traceability. The design remains under review and will be validated as features are implemented.

1.2 Scope

BloodConnect serves one blood-bank branch, donors, staff, requesters, and an administrator. It covers registration, eligibility, stock and expiry alerts, requests, issue, appointments, camps, notifications, and reports. Payments, national-registry integration, production cloud design, and native mobile apps are outside this phase.

1.3 Audience

Team 5 uses this document for implementation and testing; faculty and operational stakeholders use it to review coverage and workflows. Each member owns an end-to-end feature and supports a cross-cutting responsibility.

1.4 Definitions

Key terms: RBAC controls access by role; JWT is the proposed token format; DRF is Django REST Framework; ORM maps application objects to database records; SMTP supports email; UUIDs identify records; and SRS/STP denote the requirements and test-plan documents.

2. Document Overview

2.1 How to use this document

Section 3 covers architecture, components, and security. Section 4 covers detailed design, UML, APIs, errors, and UX. Section 5 contains references and tools. Shared IDs link the SAD to the STP.

2.2 Related Documents

1. Team5_BloodBankManagementSystem_Synopsis_Final.pdf - approved project scope, ownership, six-sprint plan and technology choices.
2. Team5_BloodBankManagementSystem_SRS.pdf - SRS v1.1; FR-001-FR-023, NFR-001-NFR-004, SEC-001-SEC-006, safety and business rules.
3. BloodConnect Software Test Plan v1.1 - security validation, functional/non-functional tests and traceability.
4. SAD_Template.docx and Test_Plan_Template for SE.docx - required document structure.
5. SE_Mini_Project_Deliverables_Part-1.pdf - current submission requirements.
6. Coffee Kiosk exemplar and SE mini-project guidelines - completeness and evidence expectations; example-project features are not copied into BloodConnect.

3. Architecture

3.1 Goals & Constraints

7. Keep stock and issue records consistent. An approved request must use matching, available, non-expired blood, and one unit cannot be issued twice.
8. Apply the SRS performance target: 95% of inventory/search responses within two seconds under a defined test load. Availability is at least 99% during the defined operating window, excluding planned maintenance.
9. Keep registration within five primary screens. Eligibility and shelf-life rules must remain configurable.
10. Use the committed stack: Django/DRF, Node.js notifications, HTML5/CSS3/JavaScript, MySQL or PostgreSQL, and Docker. The database engine and exact versions remain to be finalized.
11. Work within six Agile sprints. Keep each member responsible for a complete feature, with API review, database, UI/UX and testing/CI as secondary responsibilities.
12. Protect health information and enforce permissions on the server, not just through hidden UI controls.

3.2 Stakeholders & Concerns

Primary stakeholders are donors, blood-bank staff, requesters, administrators, faculty reviewers, and the delivery team. The design prioritizes clear workflows, safety controls, privacy, traceability, and maintainable implementation.

3.3 Component (UML) Diagram

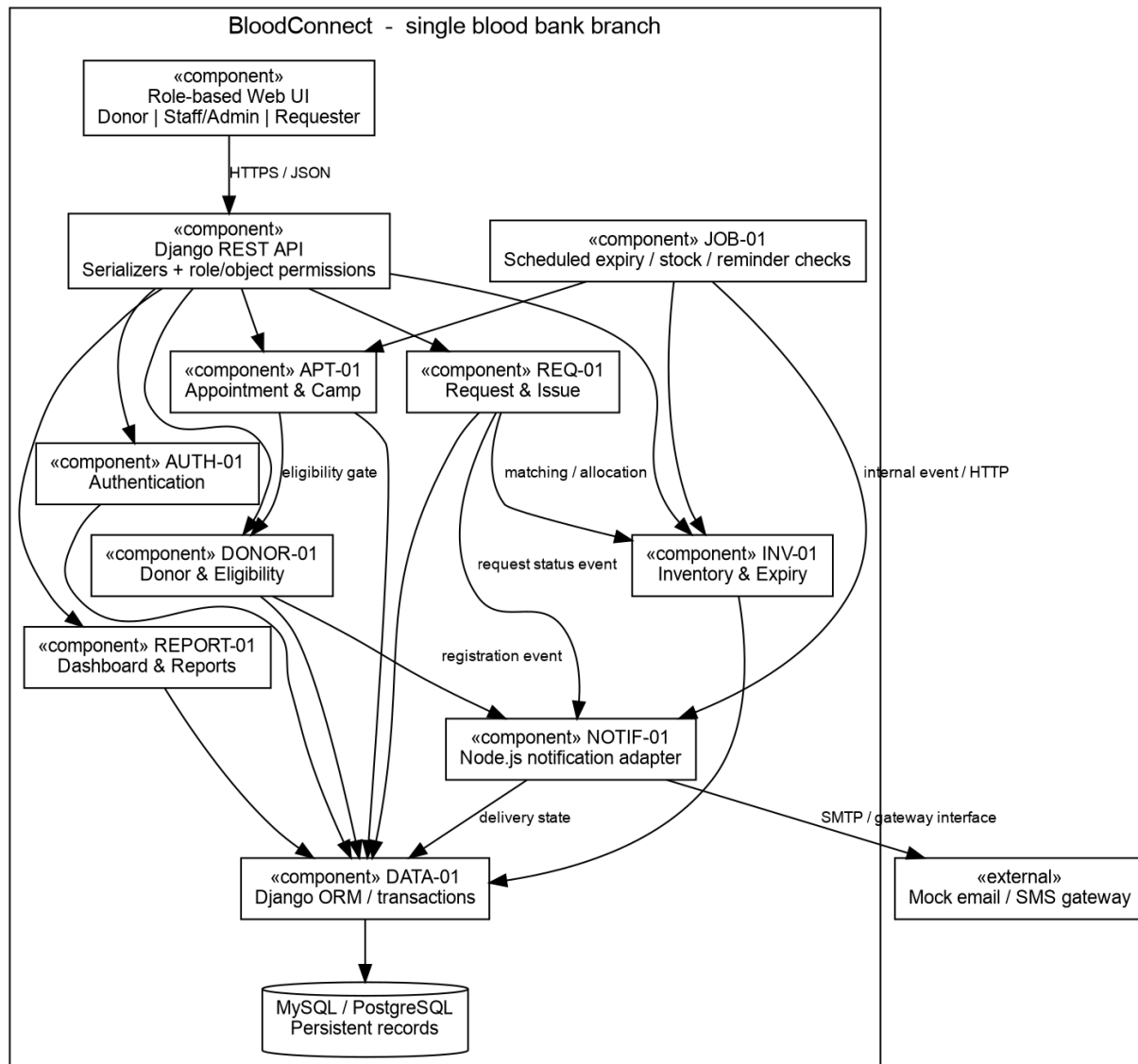


Figure 3.1 - BloodConnect component diagram. Directed connectors show API/data/service dependencies.

The component diagram connects the web interface, backend modules, database, and notification service. External delivery remains separate from transactional records such as blood issue.

3.4 Component Descriptions

ID / Component	Responsibility	Owner
WEB-01 / Role-based web UI	Donor portal, Staff/Admin dashboard and Hospital/Requester portal	Feature owner; Ch Yashwitha supports UI/UX
AUTH-01 / Authentication	Registration, login, salted password hashing, token/session checks, role permissions	Ankitha; shared permission review

ID / Component	Responsibility	Owner
DONOR-01 / Donor & Eligibility	Profiles, configurable screening, search by group/location/eligibility	Ankitha
INV-01 / Inventory & Expiry	Eligible-donor intake, expiry, available stock, threshold checks	Ch Yashwitha
REQ-01 / Request & Issue	Urgency queue, approval/rejection, matching-unit issue, status transitions	Arnav Chaudhary
APT-01 / Appointments & Camps	Camp and walk-in slots, eligibility gate, capacity locking	Anumula Arya
NOTIF-01 / Notifications	In-app records plus mocked email/SMS delivery and retry state	Anumula Arya
REPORT-01 / Dashboard & Reports	Stock, pending requests, upcoming camps and date-range aggregates	Anumula Arya
DATA-01 / ORM & persistence	Entity integrity, transactions, indexing and query access	Each feature owner; Arnav supports DB
JOB-01 / Scheduled checks	Expiry, stock threshold and reminder scheduling	Inventory and scheduling owners

3.5 Chosen Architecture Pattern and Rationale

The layered backend routes browser requests through REST controllers, services, ORM, and the database. Business rules stay in services for isolated testing. Node.js handles notifications separately, and Docker standardizes local and CI environments.

Decision	Choice and reason	Alternative / implication
ADR-01	Layered modular Django core with REST interfaces	A microservice per feature would add deployment and transaction complexity for one branch
ADR-02	Django ORM with database transactions and row locks for issue/booking	UI-only validation cannot protect simultaneous requests
ADR-03	Node.js notification adapter called after successful business commit	Gateway failure must not undo a valid blood issue; durable event/delivery records prevent silent loss
ADR-04	Read available stock from eligible unit state rather than maintain an independent unguarded counter	Reduces drift; indexing and query optimization are checked against NFR-001
ADR-05	Expiry computed from configurable component shelf-life settings	Shelf-life values will be approved for each component

3.6 Technology Stack & Data Stores

Area	Technology / store	Status
Backend	Python, Django, Django REST Framework	Committed in synopsis/SRS; versions TBD
Auxiliary service	Node.js notifications/alerts	Committed; versions TBD
Frontend	HTML5, CSS3, JavaScript; responsive browser UI	Committed
Database	MySQL or PostgreSQL; Django ORM	Engine decision pending; test chosen engine for locking/constraints
Delivery	SMTP email and SMS gateway/mock	Development/testing uses mock
Environment	Docker; Git + GitHub; Jira/GitHub Projects	Committed
CI/CD and review	Jenkins or GitHub Actions, code review, security/static checks	Tool choice per synopsis; capture execution evidence later
Testing	Django Test Framework and Postman; load/static tools as needed	Core tools from synopsis; additional tools proposed

The database stores users, donor screening, units, requests, issue records, camps, slots, appointments, notifications, configuration, and audits. Role-filtered queries protect health data; deployment variables hold credentials and signing keys.

3.7 Risks & Mitigations

Risk	Mitigation
Two requests issue the same unit	Atomic transaction; lock the unit; unique issue_records.unit_id; retest concurrent requests
Two donors book the last slot	Lock the slot/camp capacity source; recount active bookings within the transaction
Late or missed alert	Persist events; track delivery per channel; retry failed mock/gateway sends
Expiry rules differ by component/storage practice	Use approved configuration; test calculated dates against the fixture, not an assumed shelf life
Undefined availability/load window	Agree measurement window and load before acceptance; record actual exclusions
Conflicting camp permissions in SRS	Use restricted provisional create/cancel permission; resolve with team/stakeholder before baseline

3.8 Traceability to Requirements

Requirement	SRS intent	Architecture / Design
FR-001	Register donor using unique email and mobile number	AUTH-01 / 4.3.1
FR-002	Authenticate all four roles using email and password	AUTH-01 / 4.3.1
FR-003	Screen eligibility using configurable age, weight and donation interval rules	DONOR-01 / 4.1 / 4.3.2
FR-004	Display eligibility result and reason	DONOR-01 / 4.1 / 4.3.2
FR-005	Search donors by blood group, location and eligibility	DONOR-01 / 4.1 / 4.3.2
FR-006	Record unit with blood group, component, collection date and quantity	INV-01 / 4.1 / 4.3.3
FR-007	Compute expiry from configured component shelf life	INV-01 / 4.1 / 4.3.3
FR-008	Maintain real-time stock by blood group and component	INV-01 / 4.1 / 4.3.3
FR-009	Flag near-expiry units and notify staff	INV-01 / 4.1 / 4.3.3
FR-010	Generate low-stock alert below configured minimum	INV-01 / 4.1 / 4.3.3
FR-011	Prevent expired or discarded unit issue	INV-01 / 4.1 / 4.3.3
FR-012	Submit request with group, component, quantity and urgency	REQ-01 / 4.2 / 4.3.4
FR-013	Route requests to staff and order pending requests by urgency	REQ-01 / 4.2 / 4.3.4
FR-014	Approve/reject request; require rejection reason	REQ-01 / 4.2 / 4.3.4
FR-015	Create issue only for approved request and matching non-expired unit	REQ-01 / 4.2 / 4.3.4
FR-016	Decrement stock on successful issue	REQ-01 / 4.2 / 4.3.4
FR-017	Track SUBMITTED, APPROVED, REJECTED, ISSUED and CLOSED	REQ-01 / 4.2 / 4.3.4
FR-018	Book camp or walk-in appointment slot	APT-01 / 4.2 / 4.3.5
FR-019	Prevent bookings above configured capacity	APT-01 / 4.2 / 4.3.5
FR-020	Send reminder at least 24 hours before appointment	NOTIF-01 / 4.3.6
FR-021	Create, update and cancel camp with date, location and capacity	APT-01 / 4.2 / 4.3.5

Requirement	SRS intent	Architecture / Design
FR-022	Show administrator stock, requests and camps dashboard	REPORT-01 / 4.3.7
FR-023	Report donations, issues and wastage for selected date range	REPORT-01 / 4.3.7

Requirement	Architecture / design response
NFR-001	INV-01/DONOR-01: indexes, filtered queries; load validation in STP
NFR-002	Docker deployment, health checks and agreed operating-window monitoring
NFR-003	WEB-01: registration in at most five primary screens
NFR-004	REQ-01/DATA-01: atomic unit issue, row lock and unit uniqueness
SEC-001	AUTH-01: Django salted password hasher
SEC-002	AUTH-01 plus all APIs: role permissions
SEC-003	All APIs: serializer validation; safe rendering and ORM parameters
SEC-004	AUTH-01: valid session/token gate
SEC-005	DONOR-01: own-record and authorized-staff query filtering
SEC-006	AUTH-01/DATA-01: security audit events
SRS 6.2 / 6.5	Inventory eligibility gate, quarantine/expiry guards, booking eligibility, multi-channel alerts

3.9 Security Architecture

Security checks apply at API, database, and notification boundaries. The server enforces identity, role, and record permissions. Token format, expiry, refresh, and revocation remain implementation decisions.

STRIDE threat	BloodConnect example	Planned control / trace
Spoofing	Stolen/invalid credentials or tampered token	Salted hashes; token/session validation; HTTPS (SEC-001, SEC-004)
Tampering	Client edits role, request status or expiry	Server-controlled fields; allowed state transitions; serializer validation (SEC-002, SEC-003)
Repudiation	Failed login or forbidden action denied later	Timestamped security audit event without passwords/tokens (SEC-006)
Information disclosure	Donor accesses another donor's health data	Object-level permissions and role-filtered output (SEC-005)
Denial of service	Oversized requests or abusive login traffic	Input length limits; proposed rate limiting; monitor failures (SEC-003; hardening proposal)
Elevation of privilege	Donor calls staff approval or role-management API	Current-role server check; no client-assigned elevated role (SEC-002)

13. Use Django's supported salted password hashing; never include plaintext passwords in DB fields, logs or responses.
14. Use HTTPS for client-server traffic. Keep internal services on a private deployment network and authenticate internal delivery calls.
15. Serializer rules check required fields, enumerations, positive quantities and valid dates. ORM parameterization and contextual output escaping address SQL injection and XSS.
16. Donors can see their own medical information; only authorized staff can see other donors' medical information. An administrator role does not automatically imply clinical-data access.

17. Audit failed login and unauthorized access with time, action, actor if known and correlation ID. Redact credentials, tokens and medical payloads.
18. Persist notification intent after a business transaction succeeds; in-app and email/SMS channels cover critical stock and expiry alerts.

4. Design

4.1 Design Overview

Each feature has a Django module: controllers validate requests and permissions, services enforce rules, and models persist data. Role-filtered lists and shared services keep behavior consistent across UI, jobs, and APIs.

Module	Depends on	End-to-end owner / support
Donor / auth	User store, eligibility configuration, notification adapter	Ankitha / API review
Inventory	Donor eligibility, shelf-life/threshold configuration, unit store	Ch Yashwitha / UI/UX
Request / issue	Inventory, request and issue stores, notifications	Arnav / database
Appointment / notification / reporting	Donor, slot/camp store, inventory and request summaries	Anumula / testing and CI

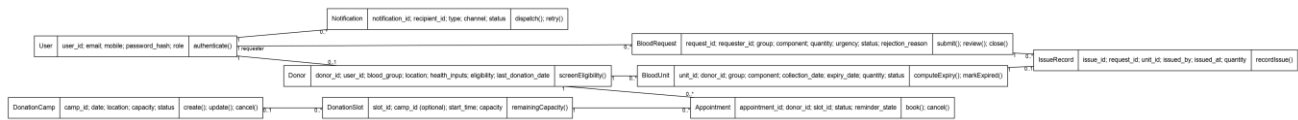


Figure 4.1 - Domain class model with association multiplicities.

DonationSlot supports camp and walk-in bookings; walk-ins omit the camp link. IssueRecord is unique by blood unit, allowing one request to use multiple units. Final health fields and visibility require stakeholder review.

4.1.1 State and transaction rules

Entity	Allowed workflow / guard
BloodRequest	SUBMITTED → APPROVED or REJECTED; APPROVED → ISSUED when fulfilled; terminal records can move to CLOSED through an agreed closure rule. Partial-issue behavior remains open.
BloodUnit	AVAILABLE → ISSUED, EXPIRED or DISCARDED; quarantine blocks issue irrespective of display state. Recheck expiry at issue time.
Appointment	Create only for currently eligible donor and available slot; cancelled bookings do not count toward capacity. Camp cancellation policy remains to be confirmed.
Issue transaction	Lock unit/request; verify approval, group/component match, remaining quantity and current unit eligibility; create issue record and update unit/request atomically. Stock reflects committed state.
Booking transaction	Lock slot and camp capacity source; check eligibility, active bookings and configured capacity; create one booking and commit.

4.1.2 Database and field layout

Entity / fields	Type / mandatory	Integrity and use
User: user_id, email, mobile, password_hash, role	UUID; email string(100); other validated strings / Y	Unique email and normalized mobile for donor registration; server-assigned roles

Entity / fields	Type / mandatory	Integrity and use
Donor: donor_id, user_id, blood_group, location, health_inputs, eligibility_status, last_donation_date	UUID/FK; enum; restricted structured fields; date / per form	One donor profile per user; configured screening inputs; location supports FR-005
BloodUnit: unit_id, donor_id, blood_group, component_type, collection_date, expiry_date, quantity, status	UUID/FK; enum; date; positive numeric / Y	FK to eligible donor; expiry from configuration; group/component/status index
BloodRequest: request_id, requester_id, blood_group, component_type, quantity, urgency_level, status, rejection_reason	UUID/FK; enums; positive numeric; text / reason conditional	Urgency index; reason required on rejection; requester ownership
IssueRecord: issue_id, request_id, unit_id, issued_by, issued_at, issued_quantity	UUID/FK; datetime; positive numeric / Y	Unique unit_id; not unique request_id; atomic issue transaction
Camp / Slot: IDs, optional camp link, scheduled time, location, capacity, status	UUID/FK; datetime; integer / Y except walk-in camp link	Capacity source locked during booking; total camp capacity also enforced
Appointment: appointment_id, donor_id, slot_id, status, reminder_state	UUID/FK; enum / Y	FKs; active donor-slot duplicate guard proposed
Notification / audit / configuration	UUID; event/channel/status/time; restricted audit context; rule key/value	Durable delivery state; no medical payload in audit logs; admin-only settings

Unspecified field lengths will follow form design. Reports summarize donations, issued quantity, and wastage. Composite indexes support donor, unit, request, and appointment queries; large lists use pagination.

4.1.3 Supporting UML views

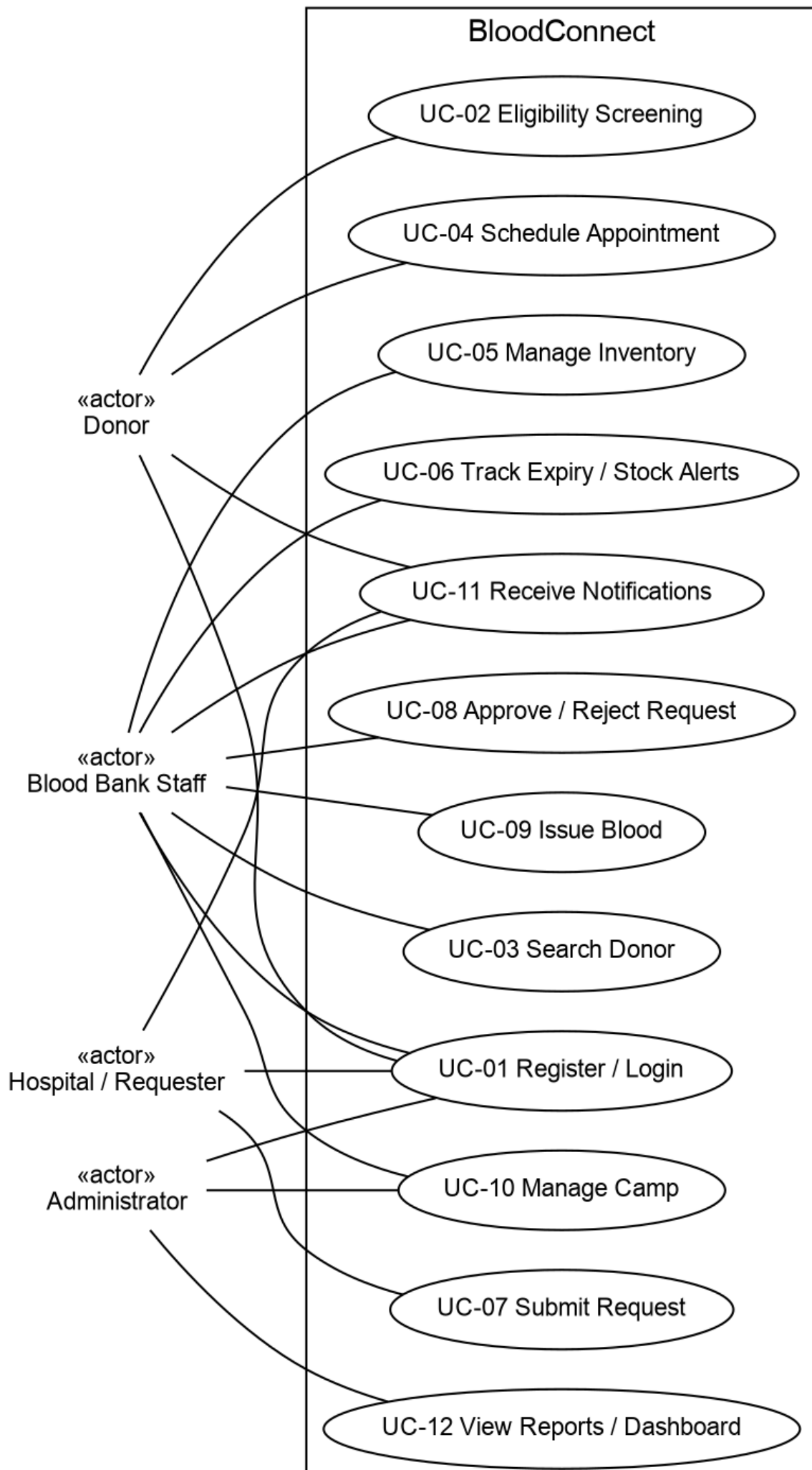


Figure 4.2 - Use-case coverage using UC-01-UC-12 from SRS Section 4.1.

The use-case diagram maps users to functions. Requests require staff review before issue; activity and sequence views show these checks.

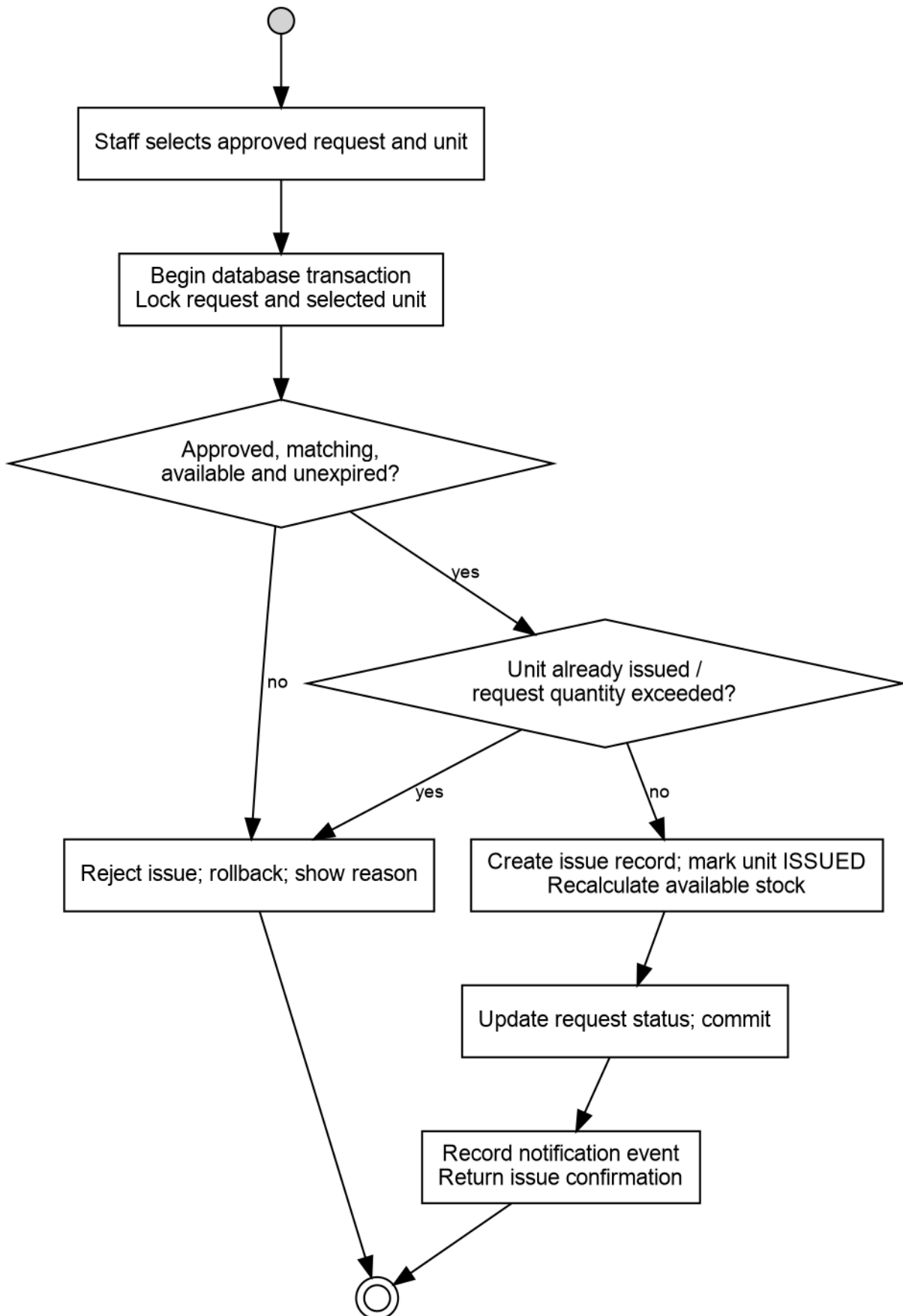


Figure 4.3 - Blood issue activity with validation and conflict branches.

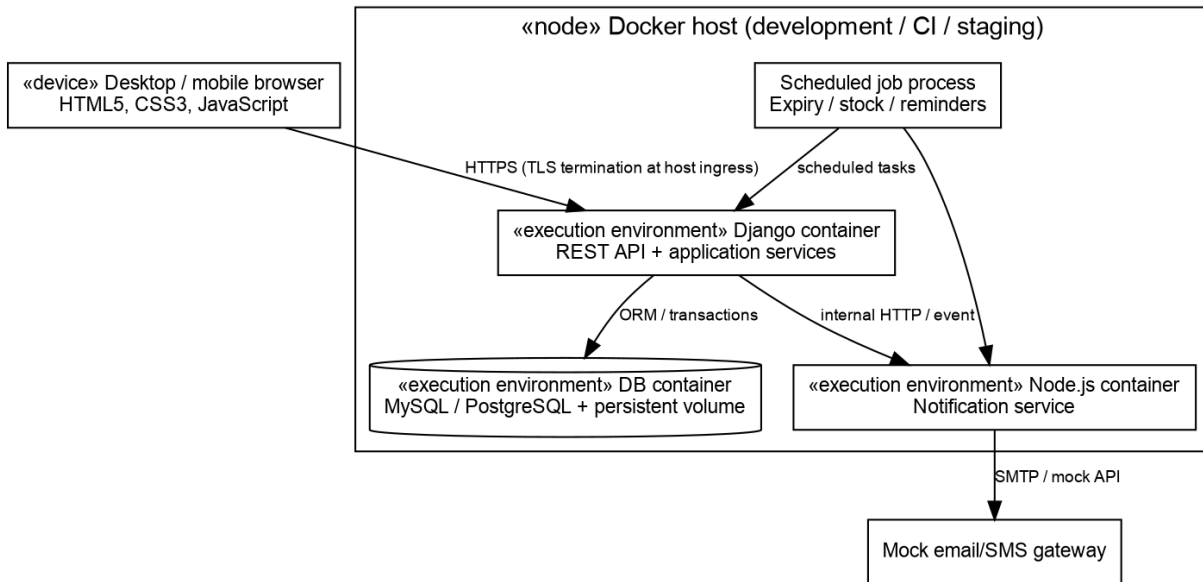
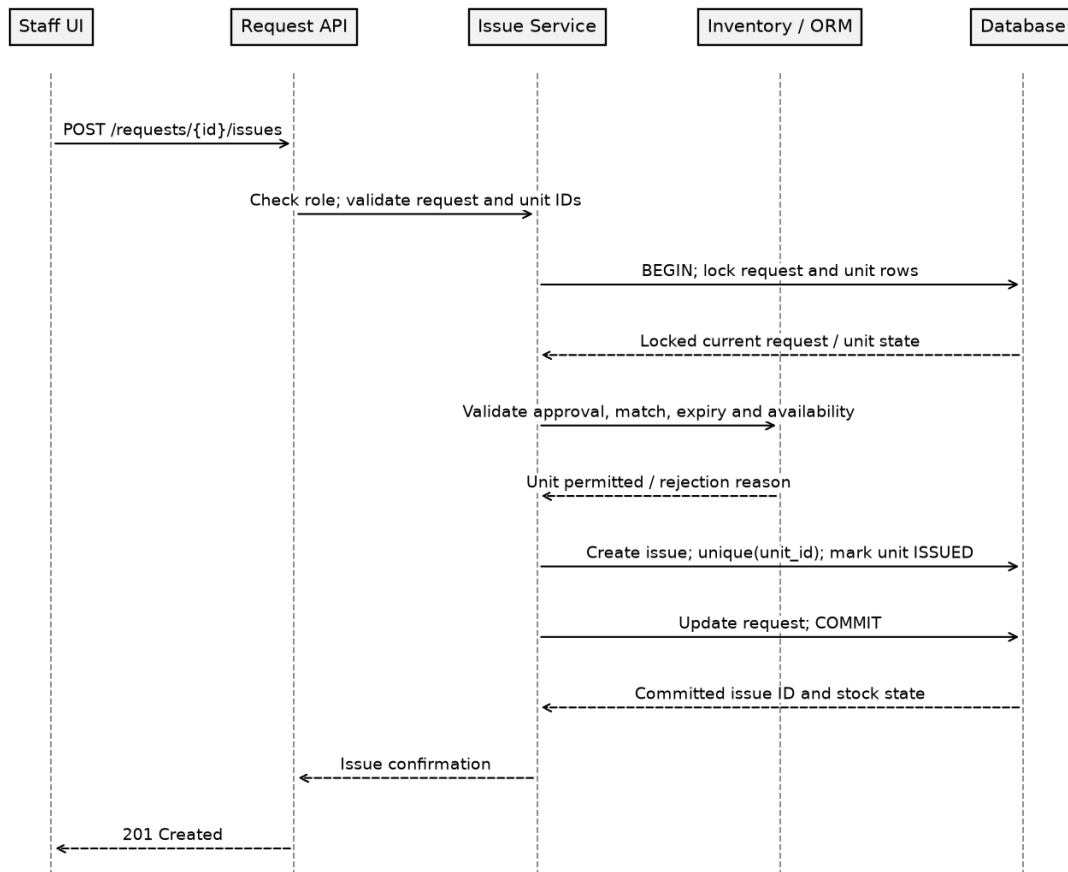


Figure 4.4 - Planned Docker deployment; exact host sizing and TLS configuration remain deployment decisions.

4.2 UML Sequence Diagrams

4.2.1 Blood request issue - UC-09



alt [validation fails or conflicting issue]: rollback and return 409/422; no stock change.
The transaction covers both issue creation and stock state. Notification is dispatched after commit.

Figure 4.5 - Atomic blood issue sequence, following staff approval (UC-08).

Issue succeeds only after transaction commit. Concurrent reuse fails without stock or record changes; later notification failure cannot reverse a completed issue.

4.2.2 Appointment booking - UC-04

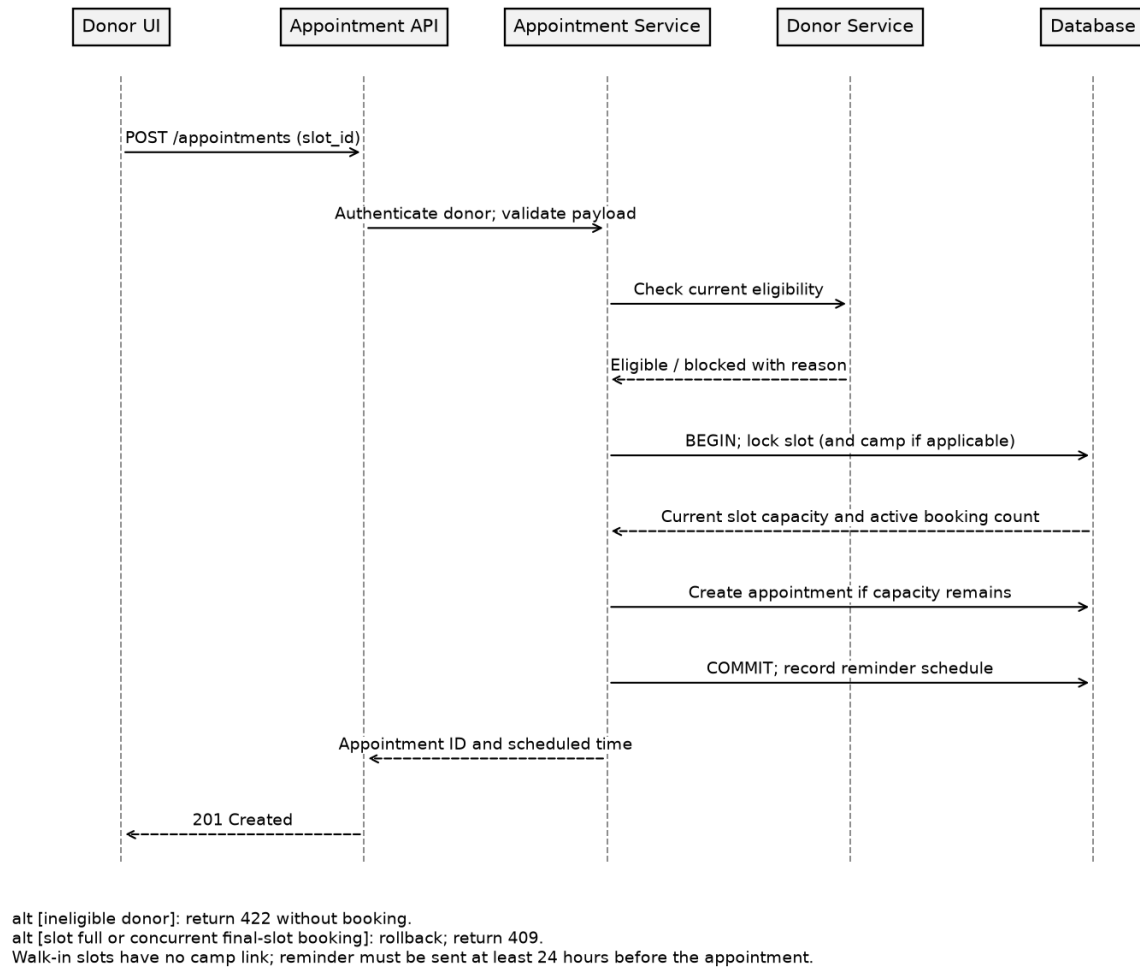


Figure 4.6 - Camp or walk-in booking with eligibility and capacity checks.

Booking reads eligibility and capacity from the database. The team must define policy for bookings made less than 24 hours ahead because the standard reminder window cannot be met.

4.3 API Design

The proposed API base path is /api/v1/. Requests and responses use JSON, with UUIDs identifying records. Registration and login are public; other endpoints require a valid session or token. The team will review these contracts before coding and update dependent modules whenever a contract changes. A separate compiled ABI is not needed because the components communicate through HTTP/JSON and Django service calls.

Method	Endpoint	Role	Input	Output / errors
POST	/auth/register	Public donor registration	Unique email/mobile, password and donor profile	201 donor_id; 400 invalid; 409 duplicate
POST	/auth/login	All roles	email, password	200 session/token + role; 401 invalid
GET	/donors	Staff	blood_group, location, eligibility filters	200 donor list; 403 forbidden
GET / PUT	/donors/{id}	Own donor / authorized staff	ID; permitted profile fields for PUT	200 profile; 403/404
POST	/donors/{id}/eligibility	Own donor / authorized staff	Configured screening fields	200 eligibility + reason; 400/403

Method	Endpoint	Role	Input	Output / errors
POST	/blood-units	Staff	donor_id, group, component, collection_date, quantity	201 unit + computed expiry; 422 ineligible donor
GET	/stock; /stock/alerts	Staff / Admin	Optional group/component filters	200 stock/alerts; 403
POST	/requests	Hospital/Requester	group, component, quantity, urgency	201 request + SUBMITTED; 400
GET	/requests; /requests/{id}	Staff; requester own; admin summary	ID/status filters	200 role-filtered queue/details; 403/404
PATCH	/requests/{id}/status	Staff	APPROVED or REJECTED; rejection reason	200 status; 400 missing reason; 422 transition
POST	/requests/{id}/issues	Staff	unit_ids; requested allocation	201 issue records; 409 conflict; 422 unsafe/nonmatching
GET	/appointment-slots	Donor / Staff / Admin	Camp or walk-in filter	200 available slot list
POST	/appointments	Donor	slot_id	201 appointment; 409 full; 422 ineligible
POST / PATCH / DELETE	/camps; /camps/{id}	See permission conflict below	date, location, capacity	201/200; 400 invalid; 403 forbidden
GET	/dashboard; /reports	Admin	Report from/to dates	200 aggregates; 400 invalid range; 403
POST	/internal/notifications	Authenticated backend service only	event_id, recipient_id, type, channels	202 recorded delivery; 400 invalid; 401 unauthorized

4.3.1 Authentication contract

The public registration form creates donor accounts only. Staff and administrator accounts are created through restricted administration. The team still needs to agree how requester accounts will be set up. A registration request can contain

```
{"email":"donor@example.test","mobile":"9000000001","password":"TestOnly@123","name":"Donor A","blood_group":"O+"}
```

A successful response contains `{"donor_id":"uuid","status":"REGISTERED"}`. Login returns the user's current role and a valid session or token. The token format, timeout and logout behavior will be decided before implementation.

4.3.2 Donor contract

Staff can search donors by blood_group, location and eligibility_status, either separately or together. POST /donors/{id}/eligibility checks the configured age, weight and donation interval and returns a result such as `{"eligible":false,"reasons":["Minimum donation interval not met"]}`. The donor and authorized staff can view the relevant health inputs. Missing screening data produces a field error rather than an assumed eligible result.

4.3.3 Inventory contract

To record a blood unit, staff must provide an eligible donor, a supported component, collection_date and a positive quantity. The service calculates expiry_date using the configured shelf life for that component. Test values are used only to check the calculation, not as clinical guidance. GET /stock includes usable, available quantities after committed changes and excludes expired, discarded and quarantined units. Stock and expiry alerts use the configured thresholds.

4.3.4 Request and issue contract

A request can contain `{"blood_group":"O+","component_type":"RBC","quantity":2,"urgency_level":"Urgent"}`. The response includes a request_id and SUBMITTED status. Staff can approve the request or reject it with a reason, for example `{"status":"REJECTED","reason":"Matching units unavailable"}`. An issue request supplies unit_ids, and the response lists the issue records, fulfilled quantity and updated request status. For this

prototype, group and component must match exactly unless an approved compatibility policy is added. Clinical cross-matching remains the responsibility of staff.

4.3.5 Appointment and camp contract

A booking request contains a slot_id for either a camp or a walk-in slot. The service checks eligibility and capacity before returning the appointment_id and scheduled_time. Camp permissions need clarification: FR-021 and Section 2.4 allow staff/admin management, while Section 6.5 limits creation and cancellation to administrators. Until this is resolved, the design allows administrators to create or cancel camps and staff to update them. The affected tests remain subject to the final decision.

4.3.6 Notification contract

The backend sends notification events with an event ID, recipient, event type, record reference, due time and channels. The Node.js service creates an in-app notification and sends critical alerts through the email/SMS mock as well. Event and channel IDs help prevent duplicate messages during retries. Appointment reminders must be sent at least 24 hours before the appointment. Messages must not contain medical details, passwords or tokens.

4.3.7 Reporting contract

The administrator dashboard shows current stock, pending requests and upcoming camps. GET /reports accepts from and to dates and returns donation, issue and wastage totals. Missing dates or a range with from later than to are rejected. Before acceptance testing, the team needs to agree the reporting timezone and how records at the start and end of the range are counted.

4.4 Error Handling, Logging & Monitoring

Condition	HTTP / handling	Evidence
Invalid field or missing required value	400; field-level error; no partial write	Validation log without sensitive payload
Missing/expired/invalid session	401; no protected response	Security audit event where relevant
Authenticated but forbidden role/object	403 or privacy-preserving 404 per agreed policy	Unauthorized-access audit record
Duplicate email/mobile or concurrent resource use	409; transaction rollback where needed	Conflict and correlated request ID
Ineligible donor, unsafe unit or invalid state	422; clear business-rule reason	Business failure count; no stock mutation
Unexpected exception / DB outage	500/503; generic client response and rollback	Restricted server trace, no secrets
Mock notification failure	Persist pending/failed delivery; retry; business commit stays valid	Delivery status and retry evidence

Errors use a common response format, for example {"error":"UNIT_UNAVAILABLE","message":"Selected unit cannot be issued","details":{},"request_id":"correlation-id"}. Failed login and unauthorized access events are logged without credentials or health data. Monitoring will cover API response times, uptime, transaction failures, expired stock and notification delivery failures. Availability must be measured across the agreed NFR-002 operating window; a brief health check is only a smoke test.

4.5 UX Design

19. Use the three interfaces from SRS Section 3.1: Donor portal, Staff/Admin dashboard and Hospital/Requester portal. They share navigation, inline validation and a consistent layout.
20. Registration collects the required details in at most five primary screens and confirms success clearly. Eligibility shows the result and reason rather than only a color.
21. Donor search exposes blood group, location and eligibility filters. Staff inventory separates the four components and makes expiry/stock alerts visible.

22. The request queue displays Emergency before Urgent before Routine. Approval/rejection uses confirmation and requires a rejection reason.
23. The issue picker omits expired, discarded and quarantined units; the API remains the authoritative guard.
24. Calendar/slot selection shows camp and walk-in options, available capacity and booking confirmation. Ineligible donors are blocked with rescreening guidance.
25. Reports use an explicit date-range selector. Status messages use text as well as color; responsive controls remain readable on mobile.

4.6 Open Issues & Next Steps

Issue	Why it matters / next action
Camp role conflict	Resolve FR-021/Section 2.4 versus Section 6.5 with team/stakeholder; update SRS, API roles and tests together
Clinical rule configuration	Approve shelf-life, screening and threshold values; use explicit test fixtures until finalized
Late bookings / reminder timing	Decide how to handle bookings made less than 24 hours ahead without violating FR-020
Issue quantity and closure	Confirm partial fulfillment, quantity unit and CLOSED transition; maintain unique unit allocation regardless
Database / authentication details	Choose MySQL or PostgreSQL, versions, token format/timeouts and revocation; exercise chosen engine's locks
Retention and access	Finalize donor health deletion/anonymization and audit retention policy as requested in SRS Section 7
Operational acceptance setup	Define test load, uptime window, timezone and report-boundary semantics before final validation
Implementation records	Link actual source files, PRs, CI results and coverage after implementation; add source references when the code is available

Implementation follows the six sprints in the synopsis: setup and login, donor features, inventory, requests and issue, scheduling and reporting, and final validation. We will update the design and test cases after each reviewed increment. Any feature outside the agreed scope needs a requirement change first.

5. Appendices

5.1 Glossary

Donor: registered individual screened before donation. Blood unit: tracked component quantity from a donation. Camp: organized donation event. Walk-in slot: donation appointment not tied to a camp. Issue record: trace of a dispatched unit against an approved request. ORM: Object-Relational Mapping. STRIDE: Spoofing, Tampering, Repudiation, Information disclosure, Denial of service and Elevation of privilege.

5.2 References

26. BloodConnect Synopsis v1.0 and SRS v1.1, Team 5, supplied project files.
27. SAD_Template.docx; Test_Plan_Template for SE.docx; SRS_Template for SE.docx, supplied TEMPLATES.zip.
28. SE Mini-Project Deliverables Part-1, PES University, current submission instructions.
29. SE Mini-Project Guidelines/Evaluation 2026 and Coffee Kiosk Student Submission Guide & Complete Exemplar, supplied reference PDFs.
30. IEEE 42010 architecture description concepts; OWASP security guidance; Django/DRF documentation as implementation references, not certification claims.

5.3 Tools

The team will use Git/GitHub, Jira or GitHub Projects, Docker, Django Test Framework, Postman and Jenkins/GitHub Actions as planned in the synopsis. UML diagrams can be maintained in draw.io, PlantUML or a similar tool. Diagram sources should be stored with the project and updated when the implementation changes.

Software Test Plan (STP) - BloodConnect

Project: BloodConnect - Blood Bank Donor, Inventory & Request Management System

Project ID: BBMS-2026-T5 | Group: Team 5

Course: UE24CS341A - Software Engineering | V SemesterVersion: 1.1 | Date: 02-10-2026 | Status: Draft for team review

Development Methodology: Agile (6 sprints)

Authors: Ankitha Lakshmmi Mohan (PES1UG24CS067); Anumula Arya (PES1UG24CS072); Arnav Chaudhary (PES1UG24CS081); Ch Yashwitha (PES1UG24CS121)

1. Introduction

Purpose: This plan describes how we will test BloodConnect against SRS v1.1. It covers what will be tested, how the tests will be run, the environment, schedule and responsibilities. The cases are planned tests. Actual results will be recorded when the corresponding features are ready.

Scope: Testing covers the single-branch web application, including donor accounts, eligibility and search, blood inventory and expiry, requests and issue, camp and walk-in appointments, notifications, dashboards and reports. Functional, security, safety and non-functional tests are included. Section 4 lists the features outside this test plan.

References: BloodConnect Synopsis v1.0, SRS v1.1 and SAD v1.1; the supplied SAD and STP templates; SE Mini-Project Deliverables Part-1; the 2026 project guidelines; and the Coffee Kiosk exemplar.

Definitions: STP means Software Test Plan; RTM means Requirements Traceability Matrix; UT, IT and ST refer to unit, integration and system tests. UAT means User Acceptance Testing, and RBAC means Role-Based Access Control. A fixture is a set of controlled test data.

2. Test Items

Item / SAD reference	What will be tested	Owner
AUTH-01	Donor registration, role login, password storage, session gate	Ankitha
DONOR-01	Configurable screening, results and all three search filters	Ankitha
INV-01	Intake, shelf life, real-time stock, expiry and low-stock checks	Ch Yashwitha
REQ-01	Submission, urgency queue, review, atomic matching-unit issue, status	Arnav Chaudhary
APT-01	Camp/walk-in slots, eligibility and capacity, camp operations	Anumula Arya
NOTIF-01 / JOB-01	Reminders, in-app plus mocked email/SMS alerts, failure/retry	Anumula Arya with inventory owner
REPORT-01	Admin dashboard and selected-range donation/issue/wastage reports	Anumula Arya
DATA-01 / WEB-01 / deployment	Constraints, transactions, UI workflows, performance and availability	All; DB and UI support per synopsis

3. Features to be Tested

Tests are based on the requirement IDs in SRS v1.1 and the design in SAD v1.1. If a requirement is unclear or conflicts with another section, the related acceptance test stays blocked until the team agrees the expected behavior.

SRS IDs	Feature / acceptance focus	Planned coverage
FR-001-FR-005	Unique email AND mobile registration; four-role login; configurable screening; result/reason; group/location/eligibility search	UC-01-UC-03; security cases
FR-006-FR-011	Unit intake; configured expiry; stock by group/component; stock/expiry alerts; unusable-unit guard	UC-05-UC-06 and UC-09
FR-012-FR-017	Valid request; staff routing/urgency; reasoned review; safe issue; stock decrement; all request states	UC-07-UC-09; closure policy review
FR-018-FR-023	Camp and walk-in booking; capacity; >=24h reminder; camp management; admin summary and reports	UC-04 and UC-10-UC-12
NFR-001-NFR-004	p95 response <2s; >=99% operating-window availability; <=5 registration screens; no simultaneous double issue	NFR-TC-01-06 plus concurrent issue test
SEC-001-SEC-006	Salted hash; RBAC; validation; token validity; health-data privacy; audit events	SEC-TC-01-20
SRS 6.2 / 6.5	Quarantine/expiry guard; ineligible booking/intake blocked; multi-channel critical alerts	UC-04, UC-05, UC-06, UC-09, UC-11

4. Features Not to be Tested

1. Payment and billing integration: explicitly outside this project phase.
2. External national blood-registry APIs and multi-branch workflows: outside the single-branch baseline.
3. Native mobile applications: only responsive browser behavior is tested.
4. Real clinical cross-match procedures or medical certification: software can enforce recorded workflow rules but does not perform clinical testing.
5. Real commercial SMS/email carrier delivery in development: mock behavior is tested; real gateway acceptance is a later integration task.
6. Final implemented source/coverage evidence before code exists: planned cases are provided, while actual results, scripts, source links and approvals remain unfilled.

5. Test Approach / Strategy

5.0.1 Test levels and types

Level / type	Approach	Evidence
Unit	Isolate eligibility, expiry, stock selection, urgency ordering and report aggregation; mock external dependencies	Django/pytest results and line/branch coverage when implemented
Integration	API + chosen DB; transaction rollback; FK/unique constraints; notifications with mocks	Request/response output, DB assertions and delivery records
System	Browser/API flows for UC-01-UC-12 with four role fixtures	Screenshots and expected-versus-actual observations

Level / type	Approach	Evidence
Acceptance / UAT	Team/faculty or representative users review the critical operational journeys	Recorded review findings and sign-off, not fabricated approval
Regression	Re-run passed automated tests after merge, schema/config or contract change	CI execution logs with build and commit reference
Performance / availability	Measure agreed load and operating window; disclose seed size and exclusions	Latency percentiles, errors, achieved throughput and uptime calculation
Usability	Observe first-time registration and responsive workflows	Primary-screen count and task notes
Security / safety	Role/object permissions, negative payloads, invalid tokens, critical workflow guards	Security findings, redacted logs and remediation/retest links

API and UI tests check the behavior visible to users. Service-level tests also cover the internal rules and branches. Boundary tests focus on age, weight, donation interval, expiry alerts and camp capacity. Negative tests cover invalid input, unsafe blood units and forbidden state changes. Concurrency tests must use separate connections to the selected MySQL or PostgreSQL database so that the actual locking behavior is tested.

5.0.2 Entry criteria

7. The sprint feature is available in a reviewed test build with its API contract and configuration recorded.
8. Isolated test DB, role accounts, synthetic donor health data and notification mocks are ready.
9. Open conflicts affecting a test's expected result are resolved or explicitly marked blocked.
10. Test runner, chosen DB locking behavior and clock controls have been checked.

5.0.3 Exit criteria

11. All applicable cases for the implemented increment have been run, with actual results and status recorded.
12. All Critical and High safety/security defects are fixed and retested.
13. The SRS acceptance conditions are met, and the RTM links tested requirements to their results.
14. Line/branch coverage is reported for implemented modules. Any numerical coverage gate is team/course-approved, agreed before execution.
15. Blocked tests and known limitations are disclosed; they do not count as passes.

5.1 Security Validation

Security tests cover the six SRS security requirements, access to individual records and HTTPS communication. They will run only on the team's development or staging system, using synthetic data. The checks focus on BloodConnect's login, permissions, input handling and donor privacy.

Requirement / objective	Validation	Cases
SEC-001: protect credentials	Review hash storage, salt behavior and redacted logging	SEC-TC-01-03
SEC-002: enforce role boundaries	Exercise donor/requester/staff forbidden operations and client role manipulation	SEC-TC-04-07
SEC-003: validate unsafe input	SQL/XSS payloads, field lengths, malformed JSON and enums	SEC-TC-08-11
SEC-004: valid sessions only	Expired, tampered and missing session/token checks	SEC-TC-12-14
SEC-005: protect donor medical data	Own-donor versus other-donor/requester/staff access	SEC-TC-15-17
SEC-006: accountability	Failed login and forbidden action events; verify no secrets in logs	SEC-TC-18-19 plus 03

Requirement / objective	Validation	Cases
SRS 3.3: secure communications	Verify configured HTTPS and non-sensitive client errors	SEC-TC-20

The security review will check authentication, authorization, unsafe input and exposure of sensitive data. Server permissions and database queries must be checked as well as the UI. Findings will be logged as defects and retested after fixes. Evidence containing health information must be restricted or redacted. Rate limits, TLS settings and token revocation still need to be finalized in the design.

6. Test Environment

Area	Planned configuration
Hardware / OS	Team laptops or staging host able to run Docker; sizing/OS recorded with test run rather than assumed
Application	Python Django/DRF backend, Node.js notification adapter, HTML5/CSS3/JavaScript frontend; versions pinned when chosen
Database	Isolated chosen MySQL/PostgreSQL engine; same transaction/constraint behavior as deployment
Browsers	Modern Chrome, Firefox and Edge; desktop and mobile-responsive viewports
Tools	Django Test Framework / Postman; pytest and a load tool such as Locust/JMeter proposed where needed; GitHub/Jira defect tracking
CI/static review	Jenkins or GitHub Actions; coverage and SonarQube or equivalent per course expectations
Gateway / clock	Controlled in-app and SMTP/SMS mocks; injectable clock for expiry/reminder/invalid-token scenarios
Data	Synthetic donors, all four roles, all components/statuses, mixed urgencies, camp/walk-in slots, known report totals
Isolation	Reset/seed DB between cases; unique fixtures; separate connections/barrier for concurrency tests

The initial screening fixtures use the SRS examples: age 18-65, weight of at least 50 kg and a minimum 90-day donation interval. Each component gets an explicit shelf-life value, L1-L4, for testing. T represents the near-expiry threshold and M the minimum stock level. These are test settings, not clinical recommendations. When testing one boundary, the other inputs should remain valid.

7. Test Schedule

Testing follows the six sprints in the synopsis. Exact start and end dates will be added once the team has agreed its sprint calendar.

Sprint	Features	Test work / milestone
1	Setup and login	Environment/fixtures; auth smoke and password/session security tests
2	Donor registration, screening, search	UC-01-03; eligibility boundaries; location search; privacy tests
3	Inventory, stock, expiry	UC-05-06; component shelf-life fixtures; alert channels; DB integration
4	Request/review/issue	UC-07-09; matching and rollback; distinct-request double-issue concurrency
5	Appointments, notifications, admin/reporting	UC-04, UC-10-12; capacity races; walk-in slots; >=24h reminders

Sprint	Features	Test work / milestone
6	Full system validation and release	All system flows; security regression; load/uptime/usability evidence; UAT and summary

Test cases and fixtures should be ready before a feature is tested. Regression tests run after each merge. Sprint 6 covers final execution, retesting and acceptance review. Availability monitoring may need to start earlier to cover the full agreed operating window.

8. Test Deliverables

16. This Software Test Plan and updated requirement/design traceability.
17. Manual case specifications in Appendices A-C; executable unit/API/system scripts when implemented.
18. Versioned fixtures and recorded configuration, load and operating-window definition.
19. Test execution logs with build/commit ID, date, tester, expected and actual result.
20. Defect reports, triage and retest evidence; security findings with redaction.
21. Unit line/branch coverage and CI/static-analysis evidence for implemented features.
22. Test Summary Report and acceptance review/sign-off after execution.

9. Roles and Responsibilities

Role / member	Functional tests	Secondary responsibility
Ankitha Lakshmmi Mohan	Donor registration, eligibility screening and donor search	API review
Anumula Arya	Appointments, notifications and admin/reporting	Testing/CI
Arnav Chaudhary	Blood request and issue workflow	Database
Ch Yashwitha	Blood inventory and stock management	UI/UX

All four members take part in integration, security and system testing. Anumula Arya coordinates testing and CI, as listed in the synopsis. Each feature owner fixes defects in their module, and another member reviews the fix and relevant regression tests. Reviewer names and signatures will be added when the review takes place.

10. Risks and Mitigation

Risk	Mitigation
Unstable feature build	Run smoke tests first; suspend only affected suite if blockers prevent meaningful execution
Database state pollution	Fresh fixtures/reset; transaction isolation; never test against live donor data
Concurrency tests pass for the wrong reason	Use real chosen engine and independent connections; synchronize requests; inspect committed records
Gateway unavailable	Use deterministic mocks and explicit failure/retry tests; do not label real carrier delivery passed
Unresolved camp permissions / reminder policy	Mark relevant cases blocked until baseline is approved; update SRS/SAD/STP together
Load/uptime window undefined	Agree and record measurement configuration; a short health check is not 99% availability proof
Missing implementation evidence	Leave code refs and actual results pending; link real commits/scripts once available

Risk	Mitigation
Late safety/security defects	Test rules early; classify as Critical/High; fix and retest before release

11. Assumptions & Dependencies

23. The submitted synopsis and SRS v1.1 remain the scope baseline; changes are reviewed through the Agile backlog.
24. Reliable site connectivity, configured eligibility/shelf-life rules and a working notification mock are available.
25. The SRS camp role contradiction is resolved before permission acceptance tests. The provisional SAD policy is resolved before the affected tests are accepted.
26. Late-booking/reminder handling, report timezone/boundaries, partial fulfillment, CLOSED transitions and retention policy are finalized before relevant tests.
27. Exact framework/database versions, load model and availability operating window are recorded before performance/reliability acceptance.
28. The present package is planning/design work. The code and execution results are not yet available, so code file links, coverage and passes remain pending.

12. Suspension & Resumption Criteria

Testing should be paused if the environment is unavailable for more than four hours, the build blocks more than 30% of runnable cases, test data is unreliable, or a safety/security issue makes further testing unsafe. If only one feature is affected, its tests can be paused while the others continue. These are test-management thresholds, not product acceptance targets.

Testing can resume once the environment and data are restored, blocking defects are fixed in a reviewed build, and smoke tests pass. The tester records the interruption, affected cases and retest scope. Cases with unresolved requirements stay blocked until the expected behavior is agreed.

13. Test Case Management & Traceability

Each case records its ID, module, requirement references, description, preconditions, steps, test data, expected result, actual result and status. The plan contains eight cases for each of the 12 SRS use cases, 20 focused security cases and six non-functional cases, giving 122 cases in total. More cases can be added as implementation reveals additional branches or failure conditions.

UT identifies service-level unit tests, IT identifies integration tests, and ST identifies end-to-end system tests. SEC-TC and NFR-TC identify security and non-functional validation. The appendices give each case its own table so the steps, data and results remain readable and easy to update.

The RTM links each numbered functional, non-functional and security requirement to the relevant component, design section and planned tests. Code references are marked TBD until the actual source files are available. Safety rules and other business rules are listed separately. The test references show planned coverage; execution evidence will be added later.

13.1 Requirements Traceability Matrix

Req ID	Brief description	Architecture ref	Design ref	Code/file ref	Test case IDs	System validation ref
FR-001	Register donor using unique email and mobile number	AUTH-01	4.3.1	TBD - implementation	IT-UC01-01, IT-UC01-02, IT-UC01-03, IT-UC01-04, ST-UC01-08	ST-UC01-08
FR-002	Authenticate all four roles using email and password	AUTH-01	4.3.1	TBD - implementation	IT-UC01-05, IT-UC01-06, IT-UC01-07, ST-UC01-08	ST-UC01-08
FR-003	Screen eligibility using configurable age, weight and donation interval rules	DONOR-01	4.1 / 4.3.2	TBD - implementation	UT-UC02-01, UT-UC02-02, IT-UC02-03, IT-UC02-04, IT-UC02-05, IT-UC02-06, ST-UC02-08	ST-UC02-08
FR-004	Display eligibility result and reason	DONOR-01	4.1 / 4.3.2	TBD - implementation	IT-UC02-07, ST-UC02-08	ST-UC02-08
FR-005	Search donors by blood group, location and eligibility	DONOR-01	4.1 / 4.3.2	TBD - implementation	UT-UC03-01, UT-UC03-02, IT-UC03-03, IT-UC03-04, IT-UC03-05, IT-UC03-06, IT-UC03-07, ST-UC03-08	ST-UC03-08
FR-006	Record unit with blood group, component, collection date and quantity	INV-01	4.1 / 4.3.3	TBD - implementation	UT-UC05-01, IT-UC05-03, ST-UC05-08	ST-UC05-08
FR-007	Compute expiry from configured component shelf life	INV-01	4.1 / 4.3.3	TBD - implementation	UT-UC05-02, IT-UC05-07, ST-UC05-08	ST-UC05-08
FR-008	Maintain real-time stock by blood group and component	INV-01	4.1 / 4.3.3	TBD - implementation	IT-UC05-05, IT-UC05-06, ST-UC05-08	ST-UC05-08
FR-009	Flag near-expiry units and notify staff	INV-01	4.1 / 4.3.3	TBD - implementation	UT-UC06-01, UT-UC06-02, IT-UC06-06, IT-UC06-07, ST-UC06-08, IT-UC11-04	ST-UC06-08
FR-010	Generate low-stock alert below configured minimum	INV-01	4.1 / 4.3.3	TBD - implementation	IT-UC06-03, IT-UC06-04, IT-UC06-06, IT-UC06-07, ST-UC06-08, IT-UC11-04	ST-UC06-08
FR-011	Prevent expired or discarded unit issue	INV-01	4.1 / 4.3.3	TBD - implementation	IT-UC05-06, IT-UC06-05, IT-UC09-03	ST-UC09-08
FR-012	Submit request with group, component, quantity and urgency	REQ-01	4.2 / 4.3.4	TBD - implementation	IT-UC07-01, IT-UC07-02, IT-UC07-03, IT-UC07-04, IT-UC07-07, ST-UC07-08	ST-UC07-08
FR-013	Route requests to staff and order pending requests by urgency	REQ-01	4.2 / 4.3.4	TBD - implementation	IT-UC07-05, IT-UC07-06, ST-UC07-08, IT-UC11-05	ST-UC07-08
FR-014	Approve/reject request; require rejection reason	REQ-01	4.2 / 4.3.4	TBD - implementation	IT-UC08-01, IT-UC08-02, IT-UC08-03, IT-UC08-04, IT-UC08-05, IT-UC08-06, IT-UC08-07, ST-UC08-08	ST-UC08-08
FR-015	Create issue only for approved request and matching non-expired unit	REQ-01	4.2 / 4.3.4	TBD - implementation	IT-UC09-01, IT-UC09-02, IT-UC09-03, IT-UC09-04, IT-UC09-07, ST-UC09-08	ST-UC09-08

Req ID	Brief description	Architecture ref	Design ref	Code/file ref	Test case IDs	System validation ref
FR-016	Decrement stock on successful issue	REQ-01	4.2 / 4.3.4	TBD - implementation	IT-UC09-01, IT-UC09-06, IT-UC09-07, ST-UC09-08	ST-UC09-08
FR-017	Track SUBMITTED, APPROVED, REJECTED, ISSUED and CLOSED	REQ-01	4.2 / 4.3.4	TBD - implementation	ST-UC07-08, IT-UC08-06, ST-UC08-08, ST-UC09-08, IT-UC11-05	ST-UC09-08 (CLOSED policy pending)
FR-018	Book camp or walk-in appointment slot	APT-01	4.2 / 4.3.5	TBD - implementation	IT-UC04-01, IT-UC04-02, IT-UC04-07, ST-UC04-08	ST-UC04-08
FR-019	Prevent bookings above configured capacity	APT-01	4.2 / 4.3.5	TBD - implementation	IT-UC04-03, IT-UC04-04, IT-UC04-05, ST-UC04-08	ST-UC04-08
FR-020	Send reminder at least 24 hours before appointment	NOTIF-01	4.3.6	TBD - implementation	UT-UC11-01, UT-UC11-02, IT-UC11-03, IT-UC11-06, IT-UC11-07, ST-UC11-08	ST-UC11-08
FR-021	Create, update and cancel camp with date, location and capacity	APT-01	4.2 / 4.3.5	TBD - implementation	IT-UC10-01, IT-UC10-02, IT-UC10-03, IT-UC10-04, IT-UC10-05, IT-UC10-06, IT-UC10-07, ST-UC10-08	ST-UC10-08
FR-022	Show administrator stock, requests and camps dashboard	REPORT-01	4.3.7	TBD - implementation	UT-UC12-01, IT-UC12-06, IT-UC12-07, ST-UC12-08	ST-UC12-08
FR-023	Report donations, issues and wastage for selected date range	REPORT-01	4.3.7	TBD - implementation	UT-UC12-02, IT-UC12-03, IT-UC12-04, IT-UC12-05, IT-UC12-06, IT-UC12-07, ST-UC12-08	ST-UC12-08
NFR-001	95% of inventory/search requests respond within 2 seconds under defined test load	INV-01 / DONOR-01	SAD 3.1 / 4.1 / 4.4	TBD - implementation	NFR-TC-01	NFR-TC-01
NFR-002	At least 99% availability in defined operating window, excluding planned maintenance	Deployment	SAD 3.1 / 4.1 / 4.4	TBD - implementation	NFR-TC-02, NFR-TC-06	NFR-TC-02
NFR-003	New donor registration takes no more than five primary screens	WEB-01	SAD 3.1 / 4.1 / 4.4	TBD - implementation	ST-UC01-08, NFR-TC-03	NFR-TC-03
NFR-004	One blood unit cannot be issued simultaneously against different approved requests	REQ-01 / DATA-01	SAD 3.1 / 4.1 / 4.4	TBD - implementation	IT-UC09-05, NFR-TC-04, NFR-TC-05	NFR-TC-04
SEC-001	Store passwords as salted hashes, never plaintext	AUTH-01 / relevant API	SAD 3.9 / 4.3	TBD - implementation	IT-UC01-06, SEC-TC-01, SEC-TC-02, SEC-TC-03	System-deployment security suite: SEC-TC-01, SEC-TC-02, SEC-TC-03
SEC-002	Prevent donor/requester access to staff/admin functions	AUTH-01 / relevant API	SAD 3.9 / 4.3	TBD - implementation	IT-UC03-07, IT-UC08-04, IT-UC10-06, IT-UC12-06, SEC-TC-04, SEC-TC-05, SEC-TC-06, SEC-TC-07	System-deployment security suite: SEC-TC-04, SEC-TC-05, SEC-TC-06, SEC-TC-07
SEC-003	Validate and sanitize user-supplied fields	AUTH-01 / relevant API	SAD 3.9 / 4.3	TBD - implementation	IT-UC01-04, IT-UC02-06, IT-UC03-06, IT-UC04-07, IT-UC05-03, IT-UC07-03, IT-UC10-04, SEC-TC-08, SEC-TC-09, SEC-TC-10, SEC-TC-11, SEC-TC-20	System-deployment security suite: SEC-TC-08, SEC-TC-09, SEC-TC-10, SEC-TC-11, SEC-TC-20

Req ID	Brief description	Architecture ref	Design ref	Code/file ref	Test case IDs	System validation ref
SEC-004	Reject expired or invalid session tokens	AUTH-01 / relevant API	SAD 3.9 / 4.3	TBD - implementation	IT-UC01-07, SEC-TC-12, SEC-TC-13, SEC-TC-14	System-deployment security suite: SEC-TC-12, SEC-TC-13, SEC-TC-14
SEC-005	Restrict donor health data to authorized staff and that donor	AUTH-01 / relevant API	SAD 3.9 / 4.3	TBD - implementation	IT-UC07-07, ST-UC11-08, SEC-TC-15, SEC-TC-16, SEC-TC-17	System-deployment security suite: SEC-TC-15, SEC-TC-16, SEC-TC-17
SEC-006	Log failed login and unauthorized access events	AUTH-01 / relevant API	SAD 3.9 / 4.3	TBD - implementation	SEC-TC-03, SEC-TC-18, SEC-TC-19	System-deployment security suite: SEC-TC-03, SEC-TC-18, SEC-TC-19

13.2 Safety, business rules and unresolved requirements

SRS reference	Check / test records
6.2: no expired/quarantined selection	IT-UC09-03; inventory stock exclusion; UI issue picker
6.2: ineligible donor cannot complete appointment	IT-UC04-06
6.2: critical alerts use more than one channel	IT-UC06-06; IT-UC11-04
6.5: only eligible donor intake	IT-UC05-04
6.5: only staff review/issue	IT-UC08-04; security requester-role tests
6.5 / FR-021: camp create/cancel roles conflict	IT-UC10-07 blocked pending approved permission matrix
FR-020: booking <24h before appointment	IT-UC11-07 blocked pending booking/reminder policy
FR-017: CLOSED / partial fulfillment	Review approved transition rules; add execution cases before acceptance
SRS 7: retention policy	Design review pending policy; add deletion/anonymization tests once approved

14. Test Metrics & Reporting

Metric	How recorded
Execution progress	Executed / applicable planned cases; also list blocked and not-run cases separately
Pass / fail	Pass and fail counts per module/level; never count blocked/not-run as passes
Requirement coverage	Requirements mapped / requirements in scope; execution coverage separately from planning coverage
Defects	Severity, owner, age, open/resolved/retested; Critical/High release blockers
Unit coverage	Line and branch coverage for actual implemented modules; record tool/build
Performance	p50/p95, endpoint mix, achieved throughput, error rate, load and dataset
Availability	Available eligible time / total eligible operating time; disclose planned maintenance and outages
Safety / concurrency	Successful allocations and resulting stock/issue rows; no duplicate unit dispatch

The team records progress during each sprint and logs defects as they are found. The final Test Summary Report is prepared after Sprint 6. Reports include the build or commit, environment, tested scope, outstanding issues and retest results. For now, Actual Result is marked "Not executed" and Result is marked "Not run". Cases that depend on an unresolved policy identify that dependency in the expected result.

15. Approvals

Role	Name	Signature / Date
Test coordination	Anumula Arya	
Development review	All Team 5 members	
Faculty reviewer	To be filled	
Stakeholder acceptance	Blood Bank Administrator / Partner Hospital Coordinator	

Reviewers will complete the approval fields after reviewing the plan. They are left blank in this draft.

Appendix A: Use-Case Test Cases

Authentication

IT-UC01-01 - Register a donor

Field	Test specification
Test Case ID / Module	IT-UC01-01 / Authentication
Requirement(s)	FR-001
Description	Register a donor
Precondition	New donor data and provisioned staff/requester/admin accounts; isolated test DB
Steps	POST registration; read persisted user/profile
Test Data	Unique email/mobile, valid password and required profile
Expected Result	201; donor account/profile created and salted hash stored
Result (Pass/Fail)	Not run

IT-UC01-02 - Reject duplicate email

Field	Test specification
Test Case ID / Module	IT-UC01-02/03 / Authentication
Requirement(s)	FR-001
Description	Reject duplicate identifiers
Precondition	New donor data and provisioned staff/requester/admin accounts; isolated test DB
Steps	Register separately with an existing email and an existing normalized mobile.
Test Data	Existing email; existing mobile
Expected Result	Each call returns 409 and creates no duplicate account.
Result (Pass/Fail)	Not run

IT-UC01-04 - Reject missing registration fields

Field	Test specification
Test Case ID / Module	IT-UC01-04 / Authentication
Requirement(s)	FR-001, SEC-003
Description	Reject missing registration fields
Precondition	New donor data and provisioned staff/requester/admin accounts; isolated test DB
Steps	Omit email, mobile and password in separate API calls
Test Data	One required field missing in each call
Expected Result	400 with field errors; no partial donor record
Result (Pass/Fail)	Not run

IT-UC01-05 - Authenticate each role

Field	Test specification
Test Case ID / Module	IT-UC01-05 / Authentication
Requirement(s)	FR-002
Description	Authenticate each role
Precondition	New donor data and provisioned staff/requester/admin accounts; isolated test DB
Steps	Login separately with all four role fixtures; inspect role-filtered session
Test Data	Correct donor/staff/requester/admin credentials
Expected Result	Each valid login succeeds; returned role matches stored role
Result (Pass/Fail)	Not run

IT-UC01-06 - Reject wrong password

Field	Test specification
Test Case ID / Module	IT-UC01-06 / Authentication
Requirement(s)	FR-002, SEC-001
Description	Reject wrong password
Precondition	New donor data and provisioned staff/requester/admin accounts; isolated test DB
Steps	Send incorrect password to login; inspect response
Test Data	Known email and wrong password
Expected Result	401; no valid session/token returned
Result (Pass/Fail)	Not run

IT-UC01-07 - Reject protected call without valid session

Field	Test specification
Test Case ID / Module	IT-UC01-07 / Authentication
Requirement(s)	FR-002, SEC-004
Description	Reject protected call without valid session
Precondition	New donor data and provisioned staff/requester/admin accounts; isolated test DB
Steps	Call own-profile API without token, then invalid/expired token
Test Data	Missing, invalid and expired token variants
Expected Result	401; protected data not exposed
Result (Pass/Fail)	Not run

ST-UC01-08 - Registration and login end to end

Field	Test specification
Test Case ID / Module	ST-UC01-08 / Authentication
Requirement(s)	FR-001, FR-002, NFR-003
Description	Registration and login end to end
Precondition	New donor data and provisioned staff/requester/admin accounts; isolated test DB
Steps	Use responsive donor UI to register, confirm and log in
Test Data	New donor fixture
Expected Result	Successful registration/login within at most five primary registration screens
Result (Pass/Fail)	Not run

Donor Eligibility

UT-UC02-01 - Accept eligibility lower boundaries

Field	Test specification
Test Case ID / Module	UT-UC02-01/02 / Donor Eligibility
Requirement(s)	FR-003
Description	Accept configured eligibility boundaries
Precondition	Donor exists; test rule fixture age 18-65 inclusive, weight >=50 kg, interval >=90 days; other inputs valid
Steps	Test exact lower limits and exact upper age boundary.
Test Data	Configured boundary values
Expected Result	Each boundary value is evaluated as eligible.
Result (Pass/Fail)	Not run

IT-UC02-03 - Reject age outside configured range

Field	Test specification
Test Case ID / Module	IT-UC02-03 / Donor Eligibility
Requirement(s)	FR-003
Description	Reject age outside configured range
Precondition	Donor exists; test rule fixture age 18-65 inclusive, weight >=50 kg, interval >=90 days; other inputs valid
Steps	Screen once below and once above limits
Test Data	Ages 17 and 66; other inputs valid
Expected Result	Ineligible with age reason
Result (Pass/Fail)	Not run

IT-UC02-04 - Reject weight below configured limit

Field	Test specification
Test Case ID / Module	IT-UC02-04 / Donor Eligibility
Requirement(s)	FR-003
Description	Reject weight below configured limit
Precondition	Donor exists; test rule fixture age 18-65 inclusive, weight >=50 kg, interval >=90 days; other inputs valid
Steps	Screen just below 50 kg
Test Data	49.9 kg; otherwise eligible
Expected Result	Ineligible with weight reason
Result (Pass/Fail)	Not run

IT-UC02-05 - Reject short donation interval

Field	Test specification
Test Case ID / Module	IT-UC02-05 / Donor Eligibility
Requirement(s)	FR-003
Description	Reject short donation interval
Precondition	Donor exists; test rule fixture age 18-65 inclusive, weight >=50 kg, interval >=90 days; other inputs valid
Steps	Screen before minimum interval
Test Data	89 days since last donation
Expected Result	Ineligible with interval reason
Result (Pass/Fail)	Not run

IT-UC02-06 - Reject invalid screening input

Field	Test specification
Test Case ID / Module	IT-UC02-06 / Donor Eligibility
Requirement(s)	FR-003, SEC-003
Description	Reject invalid screening input
Precondition	Donor exists; test rule fixture age 18-65 inclusive, weight >=50 kg, interval >=90 days; other inputs valid
Steps	Send missing weight, invalid date and negative value variants
Test Data	Missing/negative weight; malformed or future last-donation date
Expected Result	Validation error; no eligible status based on invalid input
Result (Pass/Fail)	Not run

IT-UC02-07 - Persist result and show reason

Field	Test specification
Test Case ID / Module	IT-UC02-07 / Donor Eligibility
Requirement(s)	FR-004
Description	Persist result and show reason
Precondition	Donor exists; test rule fixture age 18-65 inclusive, weight >=50 kg, interval >=90 days; other inputs valid
Steps	Screen failing donor via API; read result as donor
Test Data	Age 17 or weight 49.9
Expected Result	Result saved; response/UI gives ineligible reason
Result (Pass/Fail)	Not run

ST-UC02-08 - Change rule and rescreen end to end

Field	Test specification
Test Case ID / Module	ST-UC02-08 / Donor Eligibility
Requirement(s)	FR-003, FR-004
Description	Change rule and rescreen end to end
Precondition	Donor exists; test rule fixture age 18-65 inclusive, weight >=50 kg, interval >=90 days; other inputs valid
Steps	Admin changes test threshold; donor rescreens; inspect new status
Test Data	Test weight threshold changed from 50 to 55; donor weight 52
Expected Result	New configuration used; no hard-coded old rule; visible updated reason
Result (Pass/Fail)	Not run

Donor Search

UT-UC03-01 - Filter by blood group

Field	Test specification
Test Case ID / Module	UT-UC03-01/02 / Donor Search
Requirement(s)	FR-005
Description	Filter by blood group or location
Precondition	Staff logged in; donors seeded across groups, locations and eligibility states
Steps	Run separate blood-group and location filters.
Test Data	Valid group; valid location
Expected Result	Each result contains only matching donors.
Result (Pass/Fail)	Not run

IT-UC03-03 - Filter by eligibility

Field	Test specification
Test Case ID / Module	IT-UC03-03 / Donor Search
Requirement(s)	FR-005
Description	Filter by eligibility
Precondition	Staff logged in; donors seeded across groups, locations and eligibility states
Steps	Search eligible and ineligible separately
Test Data	Mixed eligibility records
Expected Result	Results follow requested eligibility filter
Result (Pass/Fail)	Not run

IT-UC03-04 - Combine all filters

Field	Test specification
Test Case ID / Module	IT-UC03-04 / Donor Search
Requirement(s)	FR-005
Description	Combine all filters
Precondition	Staff logged in; donors seeded across groups, locations and eligibility states
Steps	Search group/location/eligibility together
Test Data	O+, Bangalore, eligible
Expected Result	Only donors satisfying all filters returned
Result (Pass/Fail)	Not run

IT-UC03-05 - Handle zero matches

Field	Test specification
Test Case ID / Module	IT-UC03-05 / Donor Search
Requirement(s)	FR-005
Description	Handle zero matches
Precondition	Staff logged in; donors seeded across groups, locations and eligibility states
Steps	Search valid but absent combination
Test Data	Group/location combination absent in fixture
Expected Result	200 with empty list; no unrelated donor data
Result (Pass/Fail)	Not run

IT-UC03-06 - Reject invalid group filter

Field	Test specification
Test Case ID / Module	IT-UC03-06 / Donor Search
Requirement(s)	FR-005, SEC-003
Description	Reject invalid group filter
Precondition	Staff logged in; donors seeded across groups, locations and eligibility states
Steps	Call search with unsupported blood group
Test Data	blood_group=ZZ
Expected Result	400 validation error rather than unsafe query
Result (Pass/Fail)	Not run

IT-UC03-07 - Deny non-staff search

Field	Test specification
Test Case ID / Module	IT-UC03-07 / Donor Search
Requirement(s)	FR-005, SEC-002
Description	Deny non-staff search
Precondition	Staff logged in; donors seeded across groups, locations and eligibility states
Steps	Call staff search with donor and requester tokens
Test Data	Donor/requester token
Expected Result	403; no staff donor list
Result (Pass/Fail)	Not run

ST-UC03-08 - Search through staff UI

Field	Test specification
Test Case ID / Module	ST-UC03-08 / Donor Search
Requirement(s)	FR-005
Description	Search through staff UI
Precondition	Staff logged in; donors seeded across groups, locations and eligibility states
Steps	Login as staff; apply and clear filters; compare fixture
Test Data	Seeded searchable donors
Expected Result	Displayed results match filters; clearing filters restores permitted list
Result (Pass/Fail)	Not run

Appointments

IT-UC04-01 - Book a camp slot

Field	Test specification
Test Case ID / Module	IT-UC04-01 / Appointments
Requirement(s)	FR-018
Description	Book a camp slot
Precondition	Eligible donor, ineligible donor, camp slot and walk-in slot fixtures exist; capacity configured
Steps	Book available camp slot through API
Test Data	Eligible donor; camp-linked slot with space
Expected Result	201; appointment links donor and slot
Result (Pass/Fail)	Not run

IT-UC04-02 - Book a walk-in slot

Field	Test specification
Test Case ID / Module	IT-UC04-02 / Appointments
Requirement(s)	FR-018
Description	Book a walk-in slot
Precondition	Eligible donor, ineligible donor, camp slot and walk-in slot fixtures exist; capacity configured
Steps	Book available slot without camp link
Test Data	Eligible donor; walk-in slot
Expected Result	201; appointment created without a camp requirement
Result (Pass/Fail)	Not run

IT-UC04-03 - Allow final available capacity

Field	Test specification
Test Case ID / Module	IT-UC04-03/04 / Appointments
Requirement(s)	FR-019
Description	Enforce slot capacity boundary
Precondition	Eligible donor, ineligible donor, camp slot and walk-in slot fixtures exist; capacity configured
Steps	Book the final available place, then attempt one more booking.
Test Data	Slot with one place remaining
Expected Result	Final place succeeds; next request returns conflict.
Result (Pass/Fail)	Not run

IT-UC04-05 - Protect last slot under concurrency

Field	Test specification
Test Case ID / Module	IT-UC04-05 / Appointments
Requirement(s)	FR-019
Description	Protect last slot under concurrency
Precondition	Eligible donor, ineligible donor, camp slot and walk-in slot fixtures exist; capacity configured
Steps	Release two booking calls together using separate DB sessions
Test Data	Two donors; one remaining slot
Expected Result	One success; one conflict; count <= capacity
Result (Pass/Fail)	Not run

IT-UC04-06 - Block ineligible donor booking

Field	Test specification
Test Case ID / Module	IT-UC04-06 / Appointments
Requirement(s)	SRS 6.2
Description	Block ineligible donor booking
Precondition	Eligible donor, ineligible donor, camp slot and walk-in slot fixtures exist; capacity configured
Steps	Attempt camp and walk-in booking as ineligible donor
Test Data	Persisted ineligible status
Expected Result	422; no donation-linked appointment
Result (Pass/Fail)	Not run

IT-UC04-07 - Reject invalid slot identifier

Field	Test specification
Test Case ID / Module	IT-UC04-07 / Appointments
Requirement(s)	FR-018, SEC-003
Description	Reject invalid slot identifier
Precondition	Eligible donor, ineligible donor, camp slot and walk-in slot fixtures exist; capacity configured
Steps	Book unknown and malformed slot IDs
Test Data	Missing/malformed/nonexistent slot ID
Expected Result	400/404; no orphan appointment
Result (Pass/Fail)	Not run

ST-UC04-08 - Book and view confirmation end to end

Field	Test specification
Test Case ID / Module	ST-UC04-08 / Appointments
Requirement(s)	FR-018, FR-019
Description	Book and view confirmation end to end
Precondition	Eligible donor, ineligible donor, camp slot and walk-in slot fixtures exist; capacity configured
Steps	Donor chooses available slot in UI and books
Test Data	Eligible donor; available camp slot
Expected Result	Confirmation shows appointment ID/time; persisted capacity matches UI
Result (Pass/Fail)	Not run

Inventory

UT-UC05-01 - Record valid inventory unit

Field	Test specification
Test Case ID / Module	UT-UC05-01 / Inventory
Requirement(s)	FR-006
Description	Record valid inventory unit
Precondition	Staff and screened eligible/ineligible donors exist; four component shelf-life fixtures configured
Steps	POST unit intake then inspect persisted fields
Test Data	Eligible donor; O+; RBC; valid collection date; quantity=1
Expected Result	201; recorded group/component/date/quantity match input
Result (Pass/Fail)	Not run

UT-UC05-02 - Compute all component expiries

Field	Test specification
Test Case ID / Module	UT-UC05-02 / Inventory
Requirement(s)	FR-007
Description	Compute all component expiries
Precondition	Staff and screened eligible/ineligible donors exist; four component shelf-life fixtures configured
Steps	Call expiry service for each component; compare configured L
Test Data	Whole Blood, Plasma, Platelets, RBC; fixture shelf lives L1-L4
Expected Result	Each expiry equals collection date plus its configured shelf life
Result (Pass/Fail)	Not run

IT-UC05-03 - Reject invalid intake fields

Field	Test specification
Test Case ID / Module	IT-UC05-03 / Inventory
Requirement(s)	FR-006, SEC-003
Description	Reject invalid intake fields
Precondition	Staff and screened eligible/ineligible donors exist; four component shelf-life fixtures configured
Steps	POST missing component, negative/zero quantity and malformed date variants
Test Data	Invalid required fields or quantity <=0
Expected Result	400; inventory unchanged
Result (Pass/Fail)	Not run

IT-UC05-04 - Reject ineligible donor intake

Field	Test specification
Test Case ID / Module	IT-UC05-04 / Inventory
Requirement(s)	SRS 6.5
Description	Reject ineligible donor intake
Precondition	Staff and screened eligible/ineligible donors exist; four component shelf-life fixtures configured
Steps	Record unit for ineligible donor
Test Data	Donor eligibility false
Expected Result	422; no inventory unit created
Result (Pass/Fail)	Not run

IT-UC05-05 - Update stock after intake

Field	Test specification
Test Case ID / Module	IT-UC05-05 / Inventory
Requirement(s)	FR-008
Description	Update stock after intake
Precondition	Staff and screened eligible/ineligible donors exist; four component shelf-life fixtures configured
Steps	Read stock; add quantity q; reread group/component
Test Data	Matching bucket; q positive
Expected Result	Available stock increases by q after commit
Result (Pass/Fail)	Not run

IT-UC05-06 - Exclude unavailable units from stock

Field	Test specification
Test Case ID / Module	IT-UC05-06 / Inventory
Requirement(s)	FR-008, FR-011
Description	Exclude unavailable units from stock
Precondition	Staff and screened eligible/ineligible donors exist; four component shelf-life fixtures configured
Steps	Read stock with mixed status/expiry fixtures
Test Data	Available, issued, expired, discarded and quarantined units
Expected Result	Only available unexpired usable quantities counted
Result (Pass/Fail)	Not run

IT-UC05-07 - Use updated shelf-life configuration

Field	Test specification
Test Case ID / Module	IT-UC05-07 / Inventory
Requirement(s)	FR-007
Description	Use updated shelf-life configuration
Precondition	Staff and screened eligible/ineligible donors exist; four component shelf-life fixtures configured
Steps	Change fixture rule; record another unit; compare expiry
Test Data	New component shelf-life L; fresh intake
Expected Result	New unit uses current rule; no fixed 42-day assumption
Result (Pass/Fail)	Not run

ST-UC05-08 - Intake workflow end to end

Field	Test specification
Test Case ID / Module	ST-UC05-08 / Inventory
Requirement(s)	FR-006, FR-007, FR-008
Description	Intake workflow end to end
Precondition	Staff and screened eligible/ineligible donors exist; four component shelf-life fixtures configured
Steps	Staff records unit via UI then views stock and unit detail
Test Data	Eligible donor and valid unit details
Expected Result	Computed expiry and committed stock are shown consistently
Result (Pass/Fail)	Not run

Expiry and Stock Alerts

UT-UC06-01 - Flag unit at near-expiry boundary

Field	Test specification
Test Case ID / Module	UT-UC06-01/02 / Expiry and Stock Alerts
Requirement(s)	FR-009
Description	Apply near-expiry boundary
Precondition	Staff recipient and mock in-app/email/SMS channels exist; alert thresholds configured
Steps	Evaluate units exactly at and just outside the alert window.
Test Data	Boundary and boundary+1 dates
Expected Result	Only the unit inside the configured window is flagged.
Result (Pass/Fail)	Not run

IT-UC06-03 - Alert below low-stock minimum

Field	Test specification
Test Case ID / Module	IT-UC06-03/04 / Expiry and Stock Alerts
Requirement(s)	FR-010
Description	Apply low-stock threshold
Precondition	Staff recipient and mock in-app/email/SMS channels exist; alert thresholds configured
Steps	Evaluate stock below and exactly at the configured minimum.
Test Data	Minimum-1 and minimum values
Expected Result	Alert appears only below the minimum.
Result (Pass/Fail)	Not run

IT-UC06-05 - Exclude expired unit and update availability

Field	Test specification
Test Case ID / Module	IT-UC06-05 / Expiry and Stock Alerts
Requirement(s)	FR-011, SRS 6.5
Description	Exclude expired unit and update availability
Precondition	Staff recipient and mock in-app/email/SMS channels exist; alert thresholds configured
Steps	Advance test clock beyond expiry; run check and issue picker query
Test Data	Unit expired before current test time
Expected Result	Unit not selectable; expired/discarded lifecycle applied per policy
Result (Pass/Fail)	Not run

IT-UC06-06 - Deliver critical alert over two channels

Field	Test specification
Test Case ID / Module	IT-UC06-06 / Expiry and Stock Alerts
Requirement(s)	FR-009, FR-010, SRS 6.2
Description	Deliver critical alert over two channels
Precondition	Staff recipient and mock in-app/email/SMS channels exist; alert thresholds configured
Steps	Trigger one stock and one expiry alert; inspect delivery records
Test Data	In-app plus email or SMS mock
Expected Result	Both required channels attempted/recorded; correct staff recipient
Result (Pass/Fail)	Not run

IT-UC06-07 - Recover from gateway failure

Field	Test specification
Test Case ID / Module	IT-UC06-07 / Expiry and Stock Alerts
Requirement(s)	FR-009, FR-010
Description	Recover from gateway failure
Precondition	Staff recipient and mock in-app/email/SMS channels exist; alert thresholds configured
Steps	Fail external mock; trigger alert; restore mock and retry
Test Data	Gateway timeout/failure
Expected Result	In-app alert remains; failed channel persisted and retried without silent loss
Result (Pass/Fail)	Not run

ST-UC06-08 - Display alerts end to end

Field	Test specification
Test Case ID / Module	ST-UC06-08 / Expiry and Stock Alerts
Requirement(s)	FR-009, FR-010
Description	Display alerts end to end
Precondition	Staff recipient and mock in-app/email/SMS channels exist; alert thresholds configured
Steps	Staff opens alerts after scheduled checks
Test Data	Near-expiry and low-stock fixtures
Expected Result	UI identifies affected unit/bucket and current condition
Result (Pass/Fail)	Not run

Request Submission

IT-UC07-01 - Submit valid request

Field	Test specification
Test Case ID / Module	IT-UC07-01 / Request Submission
Requirement(s)	FR-012
Description	Submit valid request
Precondition	Requester, staff and inventory fixtures exist; requester has valid session
Steps	POST request; inspect persisted record
Test Data	O+, RBC, quantity=2, Urgent
Expected Result	201; unique request ID and SUBMITTED status
Result (Pass/Fail)	Not run

IT-UC07-02 - Reject missing mandatory fields

Field	Test specification
Test Case ID / Module	IT-UC07-02/03 / Request Submission
Requirement(s)	FR-012, SEC-003
Description	Reject missing or invalid request fields
Precondition	Requester, staff and inventory fixtures exist; requester has valid session
Steps	Omit mandatory fields and submit zero/negative quantity variants.
Test Data	Missing fields; zero/negative quantity
Expected Result	Each request returns 400 with no partial record.
Result (Pass/Fail)	Not run

IT-UC07-04 - Validate urgency values

Field	Test specification
Test Case ID / Module	IT-UC07-04 / Request Submission
Requirement(s)	FR-012
Description	Validate urgency values
Precondition	Requester, staff and inventory fixtures exist; requester has valid session
Steps	Submit Routine, Urgent, Emergency then unsupported urgency
Test Data	Allowed values and Unknown
Expected Result	Allowed values accepted; unsupported value rejected
Result (Pass/Fail)	Not run

IT-UC07-05 - Route request to staff review

Field	Test specification
Test Case ID / Module	IT-UC07-05 / Request Submission
Requirement(s)	FR-013
Description	Route request to staff review
Precondition	Requester, staff and inventory fixtures exist; requester has valid session
Steps	Submit requester request; inspect staff queue and notification intent
Test Data	New valid request
Expected Result	Pending request visible to staff; notification intent recorded
Result (Pass/Fail)	Not run

IT-UC07-06 - Order pending requests by urgency

Field	Test specification
Test Case ID / Module	IT-UC07-06 / Request Submission
Requirement(s)	FR-013
Description	Order pending requests by urgency
Precondition	Requester, staff and inventory fixtures exist; requester has valid session
Steps	Seed mixed pending requests; fetch staff queue
Test Data	Routine, Emergency, Urgent
Expected Result	Emergency precedes Urgent precedes Routine; tie order documented
Result (Pass/Fail)	Not run

IT-UC07-07 - Restrict requester to own requests

Field	Test specification
Test Case ID / Module	IT-UC07-07 / Request Submission
Requirement(s)	FR-012, SEC-005
Description	Restrict requester to own requests
Precondition	Requester, staff and inventory fixtures exist; requester has valid session
Steps	Requester A attempts to read Requester B request
Test Data	Different requester IDs
Expected Result	Forbidden/not-found response; no unrelated requester record exposed
Result (Pass/Fail)	Not run

ST-UC07-08 - Submit and track request end to end

Field	Test specification
Test Case ID / Module	ST-UC07-08 / Request Submission
Requirement(s)	FR-012, FR-013, FR-017
Description	Submit and track request end to end
Precondition	Requester, staff and inventory fixtures exist; requester has valid session
Steps	Requester uses UI; staff opens queue; requester checks status
Test Data	Valid blood request
Expected Result	Same request ID shown; SUBMITTED visible and review queue ordered correctly
Result (Pass/Fail)	Not run

Request Review

IT-UC08-01 - Approve submitted request

Field	Test specification
Test Case ID / Module	IT-UC08-01 / Request Review
Requirement(s)	FR-014
Description	Approve submitted request
Precondition	Staff, requester and SUBMITTED request fixtures exist
Steps	Staff PATCH status to APPROVED; inspect DB
Test Data	Submitted request and staff token
Expected Result	200; status APPROVED
Result (Pass/Fail)	Not run

IT-UC08-02 - Reject request with reason

Field	Test specification
Test Case ID / Module	IT-UC08-02/03 / Request Review
Requirement(s)	FR-014
Description	Enforce rejection reason
Precondition	Staff, requester and SUBMITTED request fixtures exist
Steps	Reject once with a reason and once without one.
Test Data	Valid reason; blank reason
Expected Result	Reasoned rejection succeeds; blank reason returns 400.
Result (Pass/Fail)	Not run

IT-UC08-04 - Deny requester approval

Field	Test specification
Test Case ID / Module	IT-UC08-04 / Request Review
Requirement(s)	FR-014, SEC-002
Description	Deny requester approval
Precondition	Staff, requester and SUBMITTED request fixtures exist
Steps	Requester sends approval PATCH
Test Data	Requester token
Expected Result	403; no approval
Result (Pass/Fail)	Not run

IT-UC08-05 - Reject unknown status value

Field	Test specification
Test Case ID / Module	IT-UC08-05 / Request Review
Requirement(s)	FR-014
Description	Reject unknown status value
Precondition	Staff, requester and SUBMITTED request fixtures exist
Steps	Staff sends unsupported state
Test Data	status=UNKNOWN
Expected Result	400; no transition
Result (Pass/Fail)	Not run

IT-UC08-06 - Reject disallowed backward transition

Field	Test specification
Test Case ID / Module	IT-UC08-06 / Request Review
Requirement(s)	FR-014, FR-017
Description	Reject disallowed backward transition
Precondition	Staff, requester and SUBMITTED request fixtures exist
Steps	Attempt to approve an already issued/closed request
Test Data	ISSUED or CLOSED request
Expected Result	422; historical state not overwritten
Result (Pass/Fail)	Not run

IT-UC08-07 - Keep concurrent review consistent

Field	Test specification
Test Case ID / Module	IT-UC08-07 / Request Review
Requirement(s)	FR-014
Description	Keep concurrent review consistent
Precondition	Staff, requester and SUBMITTED request fixtures exist
Steps	Send approve/reject concurrently with separate transactions
Test Data	Same SUBMITTED request
Expected Result	One valid final review state; losing conflicting write rejected; no inconsistent history
Result (Pass/Fail)	Not run

ST-UC08-08 - Review and display outcome end to end

Field	Test specification
Test Case ID / Module	ST-UC08-08 / Request Review
Requirement(s)	FR-014, FR-017
Description	Review and display outcome end to end
Precondition	Staff, requester and SUBMITTED request fixtures exist
Steps	Staff reviews via UI; requester reloads status
Test Data	Submitted request; approval/rejection variant
Expected Result	Correct final state and rejection reason, where applicable, displayed
Result (Pass/Fail)	Not run

Blood Issue

IT-UC09-01 - Issue matching unit successfully

Field	Test specification
Test Case ID / Module	IT-UC09-01 / Blood Issue
Requirement(s)	FR-015, FR-016
Description	Issue matching unit successfully
Precondition	Staff, two approved requests and suitable unit fixtures exist; real chosen test DB supports locks
Steps	POST issue for approved request; inspect unit/issue/stock
Test Data	Matching group/component; unexpired AVAILABLE unit
Expected Result	201; one issue record; unit ISSUED; stock decreases by issued quantity
Result (Pass/Fail)	Not run

IT-UC09-02 - Block non-approved request issue

Field	Test specification
Test Case ID / Module	IT-UC09-02 / Blood Issue
Requirement(s)	FR-015
Description	Block non-approved request issue
Precondition	Staff, two approved requests and suitable unit fixtures exist; real chosen test DB supports locks
Steps	Attempt issue for SUBMITTED and REJECTED requests
Test Data	Suitable unit; unapproved requests
Expected Result	422; no issue record or stock change
Result (Pass/Fail)	Not run

IT-UC09-03 - Block expired/discarded/quarantined unit

Field	Test specification
Test Case ID / Module	IT-UC09-03 / Blood Issue
Requirement(s)	FR-011, FR-015
Description	Block expired/discarded/quarantined unit
Precondition	Staff, two approved requests and suitable unit fixtures exist; real chosen test DB supports locks
Steps	Attempt issue with each unsafe unit fixture
Test Data	Expired, DISCARDED, quarantined variants
Expected Result	422; no unsafe allocation
Result (Pass/Fail)	Not run

IT-UC09-04 - Block group or component mismatch

Field	Test specification
Test Case ID / Module	IT-UC09-04 / Blood Issue
Requirement(s)	FR-015
Description	Block group or component mismatch
Precondition	Staff, two approved requests and suitable unit fixtures exist; real chosen test DB supports locks
Steps	Attempt issue with nonmatching group then component
Test Data	Approved O+ RBC request; A+ RBC / O+ Plasma unit
Expected Result	422; no issue or stock change
Result (Pass/Fail)	Not run

IT-UC09-05 - Prevent same unit for two requests concurrently

Field	Test specification
Test Case ID / Module	IT-UC09-05 / Blood Issue
Requirement(s)	NFR-004
Description	Prevent same unit for two requests concurrently
Precondition	Staff, two approved requests and suitable unit fixtures exist; real chosen test DB supports locks
Steps	Submit concurrent issue calls against different approved requests
Test Data	Same unit_id; two request IDs
Expected Result	Exactly one successful allocation; conflicting call rejected; one unit issue record
Result (Pass/Fail)	Not run

IT-UC09-06 - Rollback failed multi-step issue

Field	Test specification
Test Case ID / Module	IT-UC09-06 / Blood Issue
Requirement(s)	FR-016
Description	Rollback failed multi-step issue
Precondition	Staff, two approved requests and suitable unit fixtures exist; real chosen test DB supports locks
Steps	Inject DB failure after issue insert but before unit/request update
Test Data	Valid request/unit; test fault injection
Expected Result	Entire transaction rolls back; stock, unit and issue state unchanged
Result (Pass/Fail)	Not run

IT-UC09-07 - Fulfill multi-unit request without request uniqueness bug

Field	Test specification
Test Case ID / Module	IT-UC09-07 / Blood Issue
Requirement(s)	FR-015, FR-016
Description	Fulfill multi-unit request without request uniqueness bug
Precondition	Staff, two approved requests and suitable unit fixtures exist; real chosen test DB supports locks
Steps	Issue two distinct valid units against quantity-two request
Test Data	Two unit IDs; one approved request
Expected Result	Distinct unit issue records permitted; request_id not constrained to one unit; stock reduced correctly
Result (Pass/Fail)	Not run

ST-UC09-08 - Issue and track status end to end

Field	Test specification
Test Case ID / Module	ST-UC09-08 / Blood Issue
Requirement(s)	FR-015, FR-016, FR-017
Description	Issue and track status end to end
Precondition	Staff, two approved requests and suitable unit fixtures exist; real chosen test DB supports locks
Steps	Staff approves then issues full request via UI; requester tracks
Test Data	Matching units for full requested quantity
Expected Result	ISSUED displayed after commit; issue trace and stock agree
Result (Pass/Fail)	Not run

Donation Camps

IT-UC10-01 - Create camp with valid details

Field	Test specification
Test Case ID / Module	IT-UC10-01 / Donation Camps
Requirement(s)	FR-021
Description	Create camp with valid details
Precondition	Admin and staff fixtures exist; provisional policy admin create/cancel, staff update pending SRS resolution
Steps	Admin POST date/location/capacity
Test Data	Future camp date; valid location; positive capacity
Expected Result	201; camp published
Result (Pass/Fail)	Not run

IT-UC10-02 - Update camp details

Field	Test specification
Test Case ID / Module	IT-UC10-02 / Donation Camps
Requirement(s)	FR-021
Description	Update camp details
Precondition	Admin and staff fixtures exist; provisional policy admin create/cancel, staff update pending SRS resolution
Steps	Authorized user PATCH permitted camp fields
Test Data	Changed location/date before bookings
Expected Result	200; valid edits stored
Result (Pass/Fail)	Not run

IT-UC10-03 - Cancel camp as administrator

Field	Test specification
Test Case ID / Module	IT-UC10-03 / Donation Camps
Requirement(s)	FR-021
Description	Cancel camp as administrator
Precondition	Admin and staff fixtures exist; provisional policy admin create/cancel, staff update pending SRS resolution
Steps	Admin cancels camp then fetches available slots
Test Data	Existing camp
Expected Result	Camp cancelled and no longer bookable; existing-booking handling needs approved policy
Result (Pass/Fail)	Not run

IT-UC10-04 - Reject invalid capacity

Field	Test specification
Test Case ID / Module	IT-UC10-04/05 / Donation Camps
Requirement(s)	FR-021, SEC-003
Description	Reject invalid camp details
Precondition	Admin and staff fixtures exist; provisional policy admin create/cancel, staff update pending SRS resolution
Steps	Submit non-positive capacity and missing date/location variants.
Test Data	Invalid capacity; missing fields
Expected Result	Each invalid request returns 400 and creates no camp.
Result (Pass/Fail)	Not run

IT-UC10-06 - Deny donor/requester camp management

Field	Test specification
Test Case ID / Module	IT-UC10-06 / Donation Camps
Requirement(s)	FR-021, SEC-002
Description	Deny donor/requester camp management
Precondition	Admin and staff fixtures exist; provisional policy admin create/cancel, staff update pending SRS resolution
Steps	Call create/update/cancel with donor and requester tokens
Test Data	Nonstaff/nonadmin token
Expected Result	403; camp unchanged
Result (Pass/Fail)	Not run

IT-UC10-07 - Check camp role conflict against approved baseline

Field	Test specification
Test Case ID / Module	IT-UC10-07 / Donation Camps
Requirement(s)	FR-021, SRS 6.5
Description	Check camp role conflict against approved baseline
Precondition	Admin and staff fixtures exist; provisional policy admin create/cancel, staff update pending SRS resolution
Steps	Review permission matrix; execute staff create/cancel once conflict is resolved
Test Data	Final approved FR-021/6.5 permission matrix
Expected Result	Keep this test blocked until the camp permissions are agreed. Record the approved policy before execution.
Result (Pass/Fail)	Not run

ST-UC10-08 - Manage camp end to end

Field	Test specification
Test Case ID / Module	ST-UC10-08 / Donation Camps
Requirement(s)	FR-021
Description	Manage camp end to end
Precondition	Admin and staff fixtures exist; provisional policy admin create/cancel, staff update pending SRS resolution
Steps	Admin creates; permitted user edits; admin cancels in UI
Test Data	Valid camp fixture
Expected Result	Changes appear in camp list; cancellation removes availability without deleting history
Result (Pass/Fail)	Not run

Notifications**UT-UC11-01 - Send reminder at least 24 hours ahead**

Field	Test specification
Test Case ID / Module	UT-UC11-01/02 / Notifications
Requirement(s)	FR-020
Description	Apply reminder timing boundary
Precondition	Donor/staff/requester fixtures; mock gateway; deterministic clock; persistent delivery records
Steps	Evaluate a reminder due at least 24 hours ahead and one not yet due.
Test Data	Due and not-due appointments
Expected Result	Only the due reminder is queued.
Result (Pass/Fail)	Not run

IT-UC11-03 - Prevent repeated job duplicate reminder

Field	Test specification
Test Case ID / Module	IT-UC11-03 / Notifications
Requirement(s)	FR-020
Description	Prevent repeated job duplicate reminder
Precondition	Donor/staff/requester fixtures; mock gateway; deterministic clock; persistent delivery records
Steps	Run same due reminder job twice
Test Data	Same appointment/event
Expected Result	One event/channel send result; no duplicate reminder
Result (Pass/Fail)	Not run

IT-UC11-04 - Send critical alerts in-app and external channel

Field	Test specification
Test Case ID / Module	IT-UC11-04 / Notifications
Requirement(s)	FR-009, FR-010, SRS 6.2
Description	Send critical alerts in-app and external channel
Precondition	Donor/staff/requester fixtures; mock gateway; deterministic clock; persistent delivery records
Steps	Dispatch stock/expiry events; inspect both channels
Test Data	Staff recipient; in-app and email/SMS
Expected Result	Required channel records present; correct alert content
Result (Pass/Fail)	Not run

IT-UC11-05 - Notify appropriate requester/staff on workflow event

Field	Test specification
Test Case ID / Module	IT-UC11-05 / Notifications
Requirement(s)	FR-013, FR-017
Description	Notify appropriate requester/staff on workflow event
Precondition	Donor/staff/requester fixtures; mock gateway; deterministic clock; persistent delivery records
Steps	Submit/review request; inspect event recipients
Test Data	Requester A request and staff review
Expected Result	Staff notified on submission; requester receives relevant status without another user's data
Result (Pass/Fail)	Not run

IT-UC11-06 - Retry failed reminder delivery

Field	Test specification
Test Case ID / Module	IT-UC11-06 / Notifications
Requirement(s)	FR-020
Description	Retry failed reminder delivery
Precondition	Donor/staff/requester fixtures; mock gateway; deterministic clock; persistent delivery records
Steps	Fail gateway for due reminder; restore and retry before deadline
Test Data	Mock failure then success
Expected Result	Failure tracked; retry delivers within timing requirement; no duplicate on success
Result (Pass/Fail)	Not run

IT-UC11-07 - Identify late-booking policy gap

Field	Test specification
Test Case ID / Module	IT-UC11-07 / Notifications
Requirement(s)	FR-020
Description	Identify late-booking policy gap
Precondition	Donor/staff/requester fixtures; mock gateway; deterministic clock; persistent delivery records
Steps	Attempt appointment booking less than 24h ahead; compare agreed policy
Test Data	Appointment only 12h ahead
Expected Result	Keep this test blocked until the booking policy is approved. An immediate reminder for a booking 12 hours away does not meet the 24-hour requirement.
Result (Pass/Fail)	Not run

ST-UC11-08 - Receive only own notifications end to end

Field	Test specification
Test Case ID / Module	ST-UC11-08 / Notifications
Requirement(s)	FR-020, SEC-005
Description	Receive only own notifications end to end
Precondition	Donor/staff/requester fixtures; mock gateway; deterministic clock; persistent delivery records
Steps	Donor logs in after due job; compare another donor account
Test Data	Distinct recipient fixtures
Expected Result	Correct reminder displayed; unrelated notifications not exposed
Result (Pass/Fail)	Not run

Dashboard and Reports

UT-UC12-01 - Show dashboard summary

Field	Test specification
Test Case ID / Module	UT-UC12-01 / Dashboard and Reports
Requirement(s)	FR-022
Description	Show dashboard summary
Precondition	Administrator and known donation/issue/wastage/camp/request fixtures exist
Steps	Admin GET dashboard; compare seeded counts
Test Data	Known stock/pending request/upcoming camp counts
Expected Result	200; all three summaries match fixture
Result (Pass/Fail)	Not run

UT-UC12-02 - Generate date-range report

Field	Test specification
Test Case ID / Module	UT/IT-UC12-01/02 / Dashboard and Reports
Requirement(s)	FR-023
Description	Generate report with valid boundaries
Precondition	Administrator and known donation/issue/wastage/camp/request fixtures exist
Steps	Run reports using valid date ranges and exact inclusive boundaries.
Test Data	Normal and boundary date ranges
Expected Result	Totals are correct and boundary records are included once.
Result (Pass/Fail)	Not run

IT-UC12-04 - Reject inverted or invalid date range

Field	Test specification
Test Case ID / Module	IT-UC12-04 / Dashboard and Reports
Requirement(s)	FR-023
Description	Reject inverted or invalid date range
Precondition	Administrator and known donation/issue/wastage/camp/request fixtures exist
Steps	Request from later than to, then malformed date
Test Data	from > to; invalid date
Expected Result	400; no misleading report
Result (Pass/Fail)	Not run

IT-UC12-05 - Return empty report safely

Field	Test specification
Test Case ID / Module	IT-UC12-05 / Dashboard and Reports
Requirement(s)	FR-023
Description	Return empty report safely
Precondition	Administrator and known donation/issue/wastage/camp/request fixtures exist
Steps	Choose valid range with no records
Test Data	Empty fixture interval
Expected Result	200 with zero/empty results, not stale totals
Result (Pass/Fail)	Not run

IT-UC12-06 - Restrict administrative report access

Field	Test specification
Test Case ID / Module	IT-UC12-06 / Dashboard and Reports
Requirement(s)	FR-022, FR-023, SEC-002
Description	Restrict administrative report access
Precondition	Administrator and known donation/issue/wastage/camp/request fixtures exist
Steps	Call dashboard/report as donor and requester
Test Data	Donor/requester tokens
Expected Result	403; operational or donor data not leaked
Result (Pass/Fail)	Not run

IT-UC12-07 - Keep reports consistent with committed issue

Field	Test specification
Test Case ID / Module	IT-UC12-07 / Dashboard and Reports
Requirement(s)	FR-022, FR-023
Description	Keep reports consistent with committed issue
Precondition	Administrator and known donation/issue/wastage/camp/request fixtures exist
Steps	Issue unit; refresh dashboard and selected-range report
Test Data	Valid issue committed; relevant reporting date
Expected Result	Stock and issue totals reflect committed transaction once
Result (Pass/Fail)	Not run

ST-UC12-08 - Dashboard/report workflow end to end

Field	Test specification
Test Case ID / Module	ST-UC12-08 / Dashboard and Reports
Requirement(s)	FR-022, FR-023
Description	Dashboard/report workflow end to end
Precondition	Administrator and known donation/issue/wastage/camp/request fixtures exist
Steps	Admin opens dashboard; selects date range; compares seeded totals
Test Data	Known operational fixture
Expected Result	UI shows correct stock, pending requests, camps and report totals
Result (Pass/Fail)	Not run

Appendix B: Security Validation Test Cases

Security

SEC-TC-01 - Salted hash storage

Field	Test specification
Test Case ID / Module	SEC-TC-01 / Security
Requirement(s)	SEC-001
Description	Salted hash storage
Precondition	Isolated test deployment; accounts for all roles; authorized access to audit/test DB
Steps	Register donor; inspect password column using authorized test DB access
Test Data	Test password
Expected Result	Supported salted hash; no plaintext password
Result (Pass/Fail)	Not run

SEC-TC-02 - Unique salt for same password

Field	Test specification
Test Case ID / Module	SEC-TC-02 / Security
Requirement(s)	SEC-001
Description	Unique salt for same password
Precondition	Isolated test deployment; accounts for all roles; authorized access to audit/test DB
Steps	Register two donors with same password; compare hashes
Test Data	Two unique emails/mobiles; same password
Expected Result	Hashes differ; both verify correctly
Result (Pass/Fail)	Not run

SEC-TC-03 - No password/token leakage

Field	Test specification
Test Case ID / Module	SEC-TC-03 / Security
Requirement(s)	SEC-001, SEC-006
Description	No password/token leakage
Precondition	Isolated test deployment; accounts for all roles; authorized access to audit/test DB
Steps	Inspect login/register responses and audit logs
Test Data	Successful and failed login
Expected Result	Password/token not present in logs or unrelated response fields
Result (Pass/Fail)	Not run

SEC-TC-04 - Deny donor inventory changes

Field	Test specification
Test Case ID / Module	SEC-TC-04 / Security
Requirement(s)	SEC-002
Description	Deny donor inventory changes
Precondition	Isolated test deployment; accounts for all roles; authorized access to audit/test DB
Steps	Donor POST blood-units
Test Data	Valid donor token
Expected Result	403; no inventory write
Result (Pass/Fail)	Not run

SEC-TC-05 - Deny requester review and issue

Field	Test specification
Test Case ID / Module	SEC-TC-05 / Security
Requirement(s)	SEC-002
Description	Deny requester review and issue
Precondition	Isolated test deployment; accounts for all roles; authorized access to audit/test DB
Steps	Requester PATCH approval and POST issue
Test Data	Requester token
Expected Result	403; request/stock unchanged
Result (Pass/Fail)	Not run

SEC-TC-06 - Deny staff user-role management

Field	Test specification
Test Case ID / Module	SEC-TC-06 / Security
Requirement(s)	SEC-002
Description	Deny staff user-role management
Precondition	Isolated test deployment; accounts for all roles; authorized access to audit/test DB
Steps	Staff attempts restricted role-setting operation
Test Data	Staff token; admin-role payload
Expected Result	403; no elevation
Result (Pass/Fail)	Not run

SEC-TC-07 - Reject client-assigned privileged registration

Field	Test specification
Test Case ID / Module	SEC-TC-07 / Security
Requirement(s)	SEC-002
Description	Reject client-assigned privileged registration
Precondition	Isolated test deployment; accounts for all roles; authorized access to audit/test DB
Steps	POST public registration with role=admin or staff
Test Data	New registration + elevated role
Expected Result	Elevated role not granted; reject field or ignore under documented contract
Result (Pass/Fail)	Not run

SEC-TC-08 - SQL injection field input

Field	Test specification
Test Case ID / Module	SEC-TC-08 / Security
Requirement(s)	SEC-003
Description	SQL injection field input
Precondition	Isolated test deployment; accounts for all roles; authorized access to audit/test DB
Steps	Send SQL metacharacter payload to login/search
Test Data	Invalid email and filter payload
Expected Result	Rejected/safely treated as data; no SQL execution or DB error leakage
Result (Pass/Fail)	Not run

SEC-TC-09 - XSS-safe profile rendering

Field	Test specification
Test Case ID / Module	SEC-TC-09 / Security
Requirement(s)	SEC-003
Description	XSS-safe profile rendering
Precondition	Isolated test deployment; accounts for all roles; authorized access to audit/test DB
Steps	Submit script-like name; view profile and donor search
Test Data	<script>alert(1)</script>
Expected Result	Rejected or contextually escaped; no script execution
Result (Pass/Fail)	Not run

SEC-TC-10 - Enforce field length limits

Field	Test specification
Test Case ID / Module	SEC-TC-10 / Security
Requirement(s)	SEC-003
Description	Enforce field length limits
Precondition	Isolated test deployment; accounts for all roles; authorized access to audit/test DB
Steps	Submit email longer than SRS string(100) limit
Test Data	101-character email
Expected Result	400; no truncated ambiguous identity
Result (Pass/Fail)	Not run

SEC-TC-11 - Reject malformed JSON and unsupported enums

Field	Test specification
Test Case ID / Module	SEC-TC-11 / Security
Requirement(s)	SEC-003
Description	Reject malformed JSON and unsupported enums
Precondition	Isolated test deployment; accounts for all roles; authorized access to audit/test DB
Steps	POST malformed request and unsupported blood group/component
Test Data	Bad JSON; group ZZ; component Unknown
Expected Result	400; no partial write or server stack trace
Result (Pass/Fail)	Not run

SEC-TC-12 - Expired token denied

Field	Test specification
Test Case ID / Module	SEC-TC-12/13 / Security
Requirement(s)	SEC-004
Description	Deny expired or tampered tokens
Precondition	Isolated test deployment; accounts for all roles; authorized access to audit/test DB
Steps	Call protected endpoints with expired and tampered tokens.
Test Data	Expired token; modified token
Expected Result	Each call returns 401 and exposes no protected data.
Result (Pass/Fail)	Not run

SEC-TC-14 - Missing token denied

Field	Test specification
Test Case ID / Module	SEC-TC-14 / Security
Requirement(s)	SEC-004
Description	Missing token denied
Precondition	Isolated test deployment; accounts for all roles; authorized access to audit/test DB
Steps	Call protected API with no authentication
Test Data	Missing session/token
Expected Result	401; no protected output
Result (Pass/Fail)	Not run

SEC-TC-15 - Donor cannot view another donor health data

Field	Test specification
Test Case ID / Module	SEC-TC-15 / Security
Requirement(s)	SEC-005
Description	Donor cannot view another donor health data
Precondition	Isolated test deployment; accounts for all roles; authorized access to audit/test DB
Steps	Donor A GET Donor B health/eligibility details
Test Data	Two donors; A token; B ID
Expected Result	403/approved privacy-preserving 404; no B medical data
Result (Pass/Fail)	Not run

SEC-TC-16 - Requester cannot view donor health data

Field	Test specification
Test Case ID / Module	SEC-TC-16 / Security
Requirement(s)	SEC-005
Description	Requester cannot view donor health data
Precondition	Isolated test deployment; accounts for all roles; authorized access to audit/test DB
Steps	Requester calls eligibility detail API
Test Data	Requester token
Expected Result	403; no donor health inputs
Result (Pass/Fail)	Not run

SEC-TC-17 - Own donor and authorized staff can access permitted health data

Field	Test specification
Test Case ID / Module	SEC-TC-17 / Security
Requirement(s)	SEC-005
Description	Own donor and authorized staff can access permitted health data
Precondition	Isolated test deployment; accounts for all roles; authorized access to audit/test DB
Steps	Call same allowed detail as donor owner and staff
Test Data	Owner token; authorized staff token
Expected Result	Permitted responses only; password/hash excluded
Result (Pass/Fail)	Not run

SEC-TC-18 - Record failed login events

Field	Test specification
Test Case ID / Module	SEC-TC-18/19 / Security
Requirement(s)	SEC-006
Description	Audit failed authentication and authorization
Precondition	Isolated test deployment; accounts for all roles; authorized access to audit/test DB
Steps	Trigger failed login and forbidden-access events.
Test Data	Wrong password; unauthorized request
Expected Result	Audit records capture time, action, actor/correlation data without secrets.
Result (Pass/Fail)	Not run

SEC-TC-20 - Verify HTTPS and sanitized errors

Field	Test specification
Test Case ID / Module	SEC-TC-20 / Security
Requirement(s)	SEC-003 / SRS 3.3
Description	Verify HTTPS and sanitized errors
Precondition	Isolated test deployment; accounts for all roles; authorized access to audit/test DB
Steps	Inspect deployed HTTPS URL/certificate; trigger controlled error
Test Data	Staging setup and invalid request
Expected Result	Client-server HTTPS works; no stack trace/secret exposure; TLS specifics recorded
Result (Pass/Fail)	Not run

Appendix C: Non-Functional Test Cases

Non-functional

NFR-TC-01 - Inventory/search response time

Field	Test specification
Test Case ID / Module	NFR-TC-01 / Non-functional
Requirement(s)	NFR-001
Description	Inventory/search response time
Precondition	Production-like isolated staging; chosen database engine; approved measurement configuration
Steps	Warm up; run proposed 50 virtual users for five minutes; record endpoint and combined latency percentiles and achieved throughput
Test Data	Seed 1,000 donors and 5,000 units; mixed stock and donor-search requests; load proposal subject to team approval
Expected Result	At least 95% of measured inventory/search requests complete within 2 seconds under the agreed load; report errors/throughput too
Result (Pass/Fail)	Not run

NFR-TC-02 - Operating-window availability

Field	Test specification
Test Case ID / Module	NFR-TC-02 / Non-functional
Requirement(s)	NFR-002
Description	Operating-window availability
Precondition	Production-like isolated staging; chosen database engine; approved measurement configuration
Steps	Agree window and exclusions; monitor externally; compute available eligible seconds / total eligible seconds
Test Data	Proposed seven-day operating-window pilot; planned maintenance excluded with recorded timestamps
Expected Result	Availability >=99% for measured window; pilot does not establish a longer unmeasured period
Result (Pass/Fail)	Not run

NFR-TC-03 - Registration usability

Field	Test specification
Test Case ID / Module	NFR-TC-03 / Non-functional
Requirement(s)	NFR-003
Description	Registration usability
Precondition	Production-like isolated staging; chosen database engine; approved measurement configuration
Steps	Observe a new user register via desktop and mobile; count primary screens
Test Data	New donor fixture; no prior use
Expected Result	Registration completed in <=5 primary screens; capture task difficulties
Result (Pass/Fail)	Not run

NFR-TC-04 - Concurrent distinct-request unit issue

Field	Test specification
Test Case ID / Module	NFR-TC-04 / Non-functional
Requirement(s)	NFR-004
Description	Concurrent distinct-request unit issue
Precondition	Production-like isolated staging; chosen database engine; approved measurement configuration
Steps	Synchronize two approved-request issue calls on separate sessions; repeat 20 times with fresh fixtures
Test Data	One available unit and two distinct approved requests per run
Expected Result	Never two successful allocations for the unit; exactly one committed issue record per fixture
Result (Pass/Fail)	Not run

NFR-TC-05 - Concurrent retry consistency

Field	Test specification
Test Case ID / Module	NFR-TC-05 / Non-functional
Requirement(s)	NFR-004
Description	Concurrent retry consistency
Precondition	Production-like isolated staging; chosen database engine; approved measurement configuration
Steps	Retry issue after simulated lost response; verify allocation records
Test Data	Same unit after first committed issue
Expected Result	No second allocation or stock decrement; documented conflict/idempotent response
Result (Pass/Fail)	Not run

NFR-TC-06 - Restart and persistence check

Field	Test specification
Test Case ID / Module	NFR-TC-06 / Non-functional
Requirement(s)	NFR-002 / SRS 6.4
Description	Restart and persistence check
Precondition	Production-like isolated staging; chosen database engine; approved measurement configuration
Steps	Restart app/container; verify persistent records and health recovery
Test Data	Committed donor/unit/request fixture; persistent DB volume
Expected Result	Committed data retained; downtime measured and not hidden from availability calculation
Result (Pass/Fail)	Not run